

BRK (00)		Cycles: 8	Size: 2 *
Implied			
Cycle	R/W	Desc.	
-X.2	Read PC:PB	** Store as OpCode.	
1.1		Inc. PC .	
1.2	Read PC:PB	Store as Operand.	
2.1		Inc. PC .	
2.2	*** Write to (S +0)	Push PB onto stack.	
3.1			
3.2	*** Write to (S -1)	Push PC .H onto stack.	
4.1			
4.2	*** Write to (S -2)	Push PC .L onto stack.	
5.1		Use Interrupt flags to decide which vector to use. RESET = \$FFFC. NMI = \$FFEA. IRQ = \$FFEE. BRK = \$FFE6.	
5.2	*** Write to (S -3)	Push P onto stack with B flag if software BRK.	
6.1		Dec. S by 4.	
6.2	Read Vector	Store to Address.L.	
7.1		Set I , unset D .	
7.2	Read Vector + 1	Store to Address.H.	
8.1		Enable writes to PC (disabled by NMI/IRQ). Copy Address to PC . Set PB to 0.	
8.2	Read PC:PB	Store as OpCode.	
+X.1			

* Concerning the BRK instruction, you should also note that although its second byte is basically a “don’t care” byte – that is, it can have any value - the BRK (and COP instruction as well) is a two-byte instruction, the second byte sometimes is used as a signature byte to determine the nature of the BRK being executed. When an RTI instruction is executed, control always returns to the second byte past the BRK opcode. — assembly-programming-manual-for-w65c816.pdf

** If NMI, IRQ, or RESET, OpCode is forced to 0 and writes to PC are disabled.

*** If handling a RESET, the writes turn into reads. The databus is populated but ignored.

ORA (01)		Cycles: 6	Size: 2
Direct Indexed Indirect (d,x) (Read)			
Cycle	R/W	Desc.	
-X.2	Read PC:PB	Store as OpCode.	
1.1	Read PC:PB	Inc. PC .	
1.2		Store as Pointer.	
2.1		Inc. PC . Set Pointer to Pointer + X + D . Address = Pointer.	
2.2		If D.L is 0, skip the next cycle.	
3.1		Address.H = Pointer.H (fixes high byte of address).	
3.2			
4.1	Read Address	Store as Pointer.	
4.2			
5.1	Read Address + 1	Store as Pointer.H.	
5.2			
6.1	Read Pointer: DB	Store as Operand. If M flag is set, skip the next cycle.	
6.2			
7.1	Read Pointer + 1: DB	Store as Operand.H.	
7.2			
8.1	Read PC:PB	Perform A = Operand. If M flag is set, set N based off (A.L & \$80) and Z based off A.L , otherwise set N based off (A.H & \$80) and Z based off A .	
8.2		Store as OpCode.	
+X.1			

COP (02)	Cycles: 8	Size: 2
Implied		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Store as Operand.
2.1		Inc. PC .
2.2	Write to (S +0)	Push PB onto stack.
3.1		Inc. PC .
3.2	Write to (S -1)	Push PC .H onto stack.
4.1		
4.2	Write to (S -2)	Push PC .L onto stack.
5.1		Set Vector to \$FFE4.
5.2	Write to (S -3)	Push P onto stack.
6.1		Dec. S by 4.
6.2	Read Vector	Store to Address.L.
7.1		Set I , unset D .
7.2	Read Vector + 1	Store to Address.H.
8.1		Copy Address to PC . Set PB to 0.
8.2	Read PC:PB	Store as OpCode.
+X.1		

ORA (03)	Cycles: 4	Size: 2
Stack Relative (Read)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1	Read PC:PB	Inc. PC .
1.2		Store as Pointer.
2.1		Inc. PC . Set Address to Pointer + S .
2.2		
3.1		
3.2	Read Address	Store as Operand. If M flag is set, skip the next cycle.
4.1	Read Address + 1	
4.2		Store as Operand.H.
5.1	Read PC:PB	Perform A = Operand. If M flag is set, set N based off (A .L & \$80) and Z based off A .L, otherwise set N based off (A .H & \$80) and Z based off A .
5.2		Store as OpCode.
+X.1		

TSB (04)	Cycles: 5	Size: 2
Direct (Read/Modify/Write)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Store as Operand. If D.L is 0, skip the next cycle.
2.1		Inc. PC .
2.2		
3.1		Inc. PC if previous cycle was skipped. Set Address to Operand + D .
3.2	Read Address	Store as Operand. If M flag is set, skip the next cycle.
4.1		
4.2	Read Address + 1	Store as Operand.H.
5.1		If M flag is set, set Z based off (A.L & Operand.L), otherwise set Z based off (A & Operand). Perform Operand = A .
5.2		If M flag is set, skip the next cycle.
6.1		
6.2	Write to Address + 1	Write Operand.H.
7.1		
7.2	Write to Address	Write Operand.L.
8.1		
8.2	Read PC:PB	Store as OpCode.
+X.1		

ORA (05)		Cycles: 3	Size: 2
Direct (Read)			
Cycle	R/W	Desc.	
-X.2	Read PC:PB	Store as OpCode.	
1.1		Inc. PC .	
1.2	Read PC:PB	Store as Operand. If D.L is 0, skip the next cycle.	
2.1		Inc. PC .	
2.2			
3.1		Inc. PC if previous cycle was skipped. Set Address to Operand + D .	
3.2	Read Address	Store as Operand. If M flag is set, skip the next cycle.	
4.1			
4.2	Read Address + 1	Store as Operand.H.	
5.1		Perform A = Operand. If M flag is set, set N based off (A.L & \$80) and Z based off A.L , otherwise set N based off (A.H & \$80) and Z based off A .	
5.2	Read PC:PB	Store as OpCode.	
+X.1			

ASL (06)	Cycles: 5	Size: 2
Direct (Read/Modify/Write)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Store as Operand. If D.L is 0, skip the next cycle.
2.1		Inc. PC .
2.2		
3.1		Inc. PC if previous cycle was skipped. Set Address to Operand + D .
3.2	Read Address	Store as Operand. If M flag is set, skip the next cycle.
4.1		
4.2	Read Address + 1	Store as Operand.H.
5.1		If M flag is set, set C based off (Operand.L & \$80), perform Operand.L <= 1, and set N based off (Operand.L & \$80) and Z based off Operand.L, otherwise set C based off (Operand.H & \$80), perform Operand <= 1, and set N based off (Operand.H & \$80) and Z based off Operand.
5.2		If M flag is set, skip the next cycle.
6.1		
6.2	Write to Address + 1	Write Operand.H.
7.1		
7.2	Write to Address	Write Operand.L.
8.1		
8.2	Read PC:PB	Store as OpCode.
+X.1		

ORA (07)	Cycles: 6	Size: 2
Direct Indirect Long (Read)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Store as Operand. If D.L is 0, skip the next cycle.
2.1		Inc. PC .
2.2		
3.1		Inc. PC if previous cycle was skipped. Set Address to Operand + D .
3.2	Read Address	Store as Pointer.
4.1		
4.2	Read Address + 1	Store as Pointer.H.
5.1		
5.2	Read Address + 2	Store as Bank.
6.1		
6.2	Read Pointer:Bank	Store as Operand. If M flag is set, skip the next cycle.
7.1		
7.2	Read Pointer + 1:Bank (With Carry)	Store as Operand.H.
8.1		Perform A = Operand. If M flag is set, set N based off (A.L & \$80) and Z based off A.L , otherwise set N based off (A.H & \$80) and Z based off A .
8.2	Read PC:PB	Store as OpCode.
+X.1		

PHP (08)	Cycles: 3	Size: 1
Implied		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1	Write to (S +0)	Inc. PC .
1.2		
2.1		Sets Operand.L to P .
2.2		Push Operand.L onto stack.
3.1		Dec. S by 1.
3.2		Read PC:PB
+X.1		Store as OpCode.

ORA (09)	Cycles: 2	Size: 3
Immediate		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Store as Operand. If M flag is set, skip the next cycle.
2.1		Inc. PC .
2.2	Read PC:PB	Store as Operand.H.
3.1		Inc. PC . Perform A = Operand. If M flag is set, set N based off (A.L & \$80) and Z based off A.L , otherwise set N based off (A.H & \$80) and Z based off A .
3.2	Read PC:PB	Store as OpCode.
+X.1		

ASL (0A)	Cycles: 2	Size: 1
Implied		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1	Read PC:PB	Inc. PC .
1.2		Discard.
2.1		If M flag is set, set C based off (A.L & \$80), perform A.L <<= 1, and set N based off (A.L & \$80) and Z based off A.L ,
2.2		otherwise set C based off (A.H & \$80), perform A <<= 1, and set N based off (A.H & \$80) and Z based off A .
+X.1	Read PC:PB	Store as OpCode.

PHD (0B)	Cycles: 4	Size: 1
Implied		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2		
2.1		
2.2	Write to (S +0)	Push D .H onto stack.
3.1		
3.2	Write to (S -1)	Push D .L onto stack.
4.1		
4.2	Read PC:PB	Store as OpCode.
+X.1		

TSB (0C)	Cycles: 6	Size: 3
Absolute (Read/Modify/Write)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Store as Address.
2.1		Inc. PC .
2.2	Read PC:PB	Store as Address.L.
3.1		Inc. PC .
3.2	Read Address: DB	Store as Operand. If M flag is set, skip the next cycle.
4.1		
4.2	Read Address + 1: DB	Store as Operand.H.
5.1		If M flag is set, set Z based off (A .L & Operand.L), otherwise set Z based off (A & Operand). Perform Operand = A .
5.2		If M flag is set, skip the next cycle.
6.1		
6.2	Write to Address + 1: DB	Write Operand.H.
7.1		
7.2	Write to Address: DB	Write Operand.L.
8.1		
8.2	Read PC:PB	Store as OpCode.
+X.1		

ORA (0D)		Cycles: 4	Size: 3
Absolute (Read)			
Cycle	R/W	Desc.	
-X.2	Read PC:PB	Store as OpCode.	
1.1		Inc. PC .	
1.2	Read PC:PB	Store as Address.	
2.1		Inc. PC .	
2.2	Read PC:PB	Store as Address.L.	
3.1		Inc. PC .	
3.2	Read Address: DB	Store as Operand. If M flag is set, skip the next cycle.	
4.1			
4.2	Read Address + 1: DB	Store as Operand.H.	
5.1		Perform A = Operand. If M flag is set, set N based off (A .L & \$80) and Z based off A .L, otherwise set N based off (A .H & \$80) and Z based off A .	
5.2	Read PC:PB	Store as OpCode.	
+X.1			

ASL (0E)	Cycles: 6	Size: 3
Absolute (Read/Modify/Write)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Store as Address.
2.1		Inc. PC .
2.2	Read PC:PB	Store as Address.L.
3.1		Inc. PC .
3.2	Read Address: DB	Store as Operand. If M flag is set, skip the next cycle.
4.1		
4.2	Read Address + 1: DB	Store as Operand.H.
5.1		If M flag is set, set C based off (Operand.L & \$80), perform Operand.L <= 1, and set N based off (Operand.L & \$80) and Z based off Operand.L, otherwise set C based off (Operand.H & \$80), perform Operand <= 1, and set N based off (Operand.H & \$80) and Z based off Operand.
5.2		If M flag is set, skip the next cycle.
6.1		
6.2	Write to Address + 1: DB	Write Operand.H.
7.1		
7.2	Write to Address: DB	Write Operand.L.
8.1		
8.2	Read PC:PB	Store as OpCode.
+X.1		

ORA (0F)	Cycles: 5	Size: 4
Absolute Long (Read)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Store as Address.
2.1		Inc. PC .
2.2	Read PC:PB	Store as Address.L.
3.1		Inc. PC .
3.2	Read PC:PB	Stores as Bank.
4.1		Inc. PC .
4.2	Read Address:Bank	Store as Operand. If M flag is set, skip the next cycle.
5.1		
5.2	Read Address + 1:Bank (With Carry)	Store as Operand.H.
6.1		Perform A = Operand. If M flag is set, set N based off (A .L & \$80) and Z based off A .L, otherwise set N based off (A .H & \$80) and Z based off A .
6.2	Read PC:PB	Store as OpCode.
+X.1		

BPL (10)	Cycles: 2	Size: 2
Relative		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC . Decide to branch if N is 0.
1.2	Read PC:PB	Store as Operand. Address = i16(i8(Operand.L)) + PC . If not branching, poll interrupts.
2.1		Inc. PC . If not branching, end (next half-cycle is 3.2).
2.2		Poll interrupts.
3.1		Set PC to Address.
3.2	Read PC:PB	Store as OpCode.
+X.1		

ORA (11)		Cycles: 5	Size: 2
Direct Indirect Indexed (d),y (Read)			
Cycle	R/W	Desc.	
-X.2	Read PC:PB	Store as OpCode.	
1.1		Inc. PC .	
1.2	Read PC:PB	Store as Operand. If D.L is 0, skip the next cycle.	
2.1		Inc. PC .	
2.2			
3.1		Inc. PC if previous cycle was skipped. Set Address to Operand + D .	
3.2	Read Address	Store as Pointer.	
4.1			
4.2	Read Address + 1	Store as Pointer.H.	
5.1		Perform Orig = Pointer. Tmp = ui32(Pointer + Y). Pointer = ui16(Tmp). If Orig.H == Pointer.H and the X flag is set, skip 2 half-cycles. Bank = DB + (Tmp >> 16).	
5.2			
6.1			
6.2	Read Pointer:Bank	Store as Operand. If M flag is set, skip the next cycle.	
7.1			
7.2	Read Pointer + 1:Bank (With Carry)	Store as Operand.H.	
8.1		Perform A = Operand. If M flag is set, set N based off (A.L & \$80) and Z based off A.L , otherwise set N based off (A.H & \$80) and Z based off A .	
8.2	Read PC:PB	Store as OpCode.	
+X.1			

ORA (12)	Cycles: 5	Size: 2
Direct Indirect (Read)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Store as Operand. If D.L is 0, skip the next cycle.
2.1		Inc. PC .
2.2		
3.1		Inc. PC if previous cycle was skipped. Set Address to Operand + D .
3.2	Read Address	Store as Pointer.
4.1		
4.2	Read Address + 1	Store as Pointer.H.
5.1		
5.2	Read PC:PB	Store as Operand. If M flag is set, skip the next cycle.
6.1		
6.2	Read Pointer + 1: DB	Store as Operand.H.
7.1		Perform A = Operand. If M flag is set, set N based off (A.L & \$80) and Z based off A.L , otherwise set N based off (A.H & \$80) and Z based off A .
7.2	Read PC:PB	Store as OpCode.
+X.1		

ORA (13)		Cycles: 7	Size: 2
Stack Relative Indirect Indexed (Read)			
Cycle	R/W	Desc.	
-X.2	Read PC:PB	Store as OpCode.	
1.1	Read PC:PB	Inc. PC .	
1.2		Store as Pointer.	
2.1		Inc. PC . Set Address to Pointer + S .	
2.2			
3.1			
3.2	Read Address	Store as Pointer.	
4.1	Read Address + 1	Store as Pointer.H.	
4.2		Perform Orig = Pointer.	
5.1		Tmp = ui32(Pointer + Y).	
		Pointer = ui16(Tmp).	
		Bank = DB + (Tmp >> 16).	
5.2	Read Pointer:Bank		
6.1			
6.2		Store as Operand. If M flag is set, skip the next cycle.	
7.1			
7.2		Store as Operand.H.	
8.1	Read Pointer + 1:Bank (With Carry)	Perform A = Operand. If M flag is set, set N based off (A .L & \$80) and Z based off A .L, otherwise set N based off (A .H & \$80) and Z based off A .	
8.2		Store as OpCode.	
+X.1			

TRB (14)	Cycles: 5	Size: 2
Direct (Read/Modify/Write)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Store as Operand. If D.L is 0, skip the next cycle.
2.1		Inc. PC .
2.2		
3.1		Inc. PC if previous cycle was skipped. Set Address to Operand + D .
3.2	Read Address	Store as Operand. If M flag is set, skip the next cycle.
4.1		
4.2	Read Address + 1	Store as Operand.H.
5.1		If M flag is set, set Z based off (A.L & Operand.L), otherwise set Z based off (A & Operand). Perform Operand &= ~ A .
5.2		If M flag is set, skip the next cycle.
6.1		
6.2	Write to Address + 1	Write Operand.H.
7.1		
7.2	Write to Address	Write Operand.L.
8.1		
8.2	Read PC:PB	Store as OpCode.
+X.1		

ORA (15)		Cycles: 4	Size: 2
Direct, X (Read)			
Cycle	R/W	Desc.	
-X.2	Read PC:PB	Store as OpCode.	
1.1	Read PC:PB	Inc. PC .	
1.2		Store as Operand.	
2.1		Inc. PC .	
2.2			
3.1		Address = Operand + X + D . If D.L is 0, skip 2 half-cycles.	
3.2			
4.1			
4.2	Read Address	Store as Operand. If M flag is set, skip the next cycle.	
5.1	Read Address + 1		
5.2		Store as Operand.H.	
6.1	Read PC:PB	Perform A = Operand. If M flag is set, set N based off (A.L & \$80) and Z based off A.L , otherwise set N based off (A.H & \$80) and Z based off A .	
6.2		Store as OpCode.	
+X.1			

ASL (16)	Cycles: 6	Size: 2
Direct, X (Read/Modify/Write)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1	Read PC:PB	Inc. PC .
1.2		Store as Operand.
2.1		Inc. PC .
2.2		
3.1		Address = Operand + X + D . If D.L is 0, skip 2 half-cycles.
3.2		
4.1		
4.2	Read Address	Store as Operand. If M flag is set, skip the next cycle.
5.1	Read Address + 1	
5.2		Store as Operand.H.
6.1		If M flag is set, set C based off (Operand.L & \$80), perform Operand.L <= 1, and set N based off (Operand.L & \$80) and Z based off Operand.L, otherwise set C based off (Operand.H & \$80), perform Operand <= 1, and set N based off (Operand.H & \$80) and Z based off Operand.
6.2		If M flag is set, skip the next cycle.
7.1	Write to Address + 1	
7.2		Write Operand.H.
8.1	Write to Address	
8.2		Write Operand.L.
9.1	Read PC:PB	
9.2		Store as OpCode.
+X.1		

ORA (17)		Cycles: 6	Size: 2
Direct Indirect Indexed Long [d],y (Read)			
Cycle	R/W	Desc.	
-X.2	Read PC:PB	Store as OpCode.	
1.1		Inc. PC .	
1.2	Read PC:PB	Store as Operand. If D.L is 0, skip the next cycle.	
2.1		Inc. PC .	
2.2			
3.1		Inc. PC if previous cycle was skipped. Set Address to Operand + D .	
3.2	Read Address	Store as Pointer.	
4.1			
4.2	Read Address + 1	Store as Pointer.H.	
5.1			
5.2	Read Address + 2	Store as Bank.	
6.1		Perform $\text{Tmp} = \text{ui32}(\text{Pointer} + \text{Y})$.	
		Pointer = $\text{ui16}(\text{Tmp})$.	
		Bank += (Tmp >> 16).	
6.2	Read Pointer:Bank	Store as Operand. If M flag is set, skip the next cycle.	
7.1			
7.2	Read Pointer + 1:Bank (With Carry)	Store as Operand.H.	
8.1		Perform $\text{A} \mid = \text{Operand}$. If M flag is set, set N based off (A.L & \$80) and Z based off A.L , otherwise set N based off (A.H & \$80) and Z based off A .	
8.2	Read PC:PB	Store as OpCode.	
+X.1			

CLC (18)	Cycles: 2	Size: 1
Implied		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Discard.
2.1		Sets the C flag to 0.
2.2	Read PC:PB	Store as OpCode.
+X.1		

ORA (19)		Cycles: 4	Size: 3
Absolute, Y (Read)			
Cycle	R/W	Desc.	
-X.2	Read PC:PB	Store as OpCode.	
1.1		Inc. PC .	
1.2	Read PC:PB	Store as Pointer.	
2.1		Inc. PC .	
2.2	Read PC:PB	Store as Pointer.H.	
3.1		Inc. PC . Perform Orig = Pointer. Tmp = ui32(Pointer + Y). Pointer = ui16(Tmp). If Orig.H == Pointer.H and the X flag is set, skip 2 half-cycles. Bank = DB + (Tmp >> 16).	
3.2			
4.1			
4.2	Read Pointer:Bank	Store as Operand. If M flag is set, skip the next cycle.	
5.1			
5.2	Read Pointer + 1:Bank (With Carry)	Store as Operand.H.	
6.1		Perform A = Operand. If M flag is set, set N based off (A .L & \$80) and Z based off A .L, otherwise set N based off (A .H & \$80) and Z based off A .	
6.2	Read PC:PB	Store as OpCode.	
+X.1			

INC (1A)		Cycles: 2	Size: 1
Implied			
Cycle	R/W		Desc.
-X.2	Read PC:PB		Store as OpCode.
1.1	Read PC:PB		Inc. PC .
1.2			Discard.
2.1			If M flag is set, perform A.L += 1, set N based off (A.L & \$80), and set Z based off A.L , otherwise perform A += 1, set N based off (A.H & \$80), and set Z based off A .
2.2			Store as OpCode.
+X.1			

TSC (1B)	Cycles: 2	Size: 1
Implied		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Discard.
2.1		Sets S to A .
2.2	Read PC:PB	Store as OpCode.
+X.1		

TRB (1C)	Cycles: 6	Size: 3
Absolute (Read/Modify/Write)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Store as Address.
2.1		Inc. PC .
2.2	Read PC:PB	Store as Address.L.
3.1		Inc. PC .
3.2	Read Address: DB	Store as Operand. If M flag is set, skip the next cycle.
4.1		
4.2	Read Address + 1: DB	Store as Operand.H.
5.1		If M flag is set, set Z based off (A .L & Operand.L), otherwise set Z based off (A & Operand). Perform Operand &= ~ A .
5.2		If M flag is set, skip the next cycle.
6.1		
6.2	Write to Address + 1: DB	Write Operand.H.
7.1		
7.2	Write to Address: DB	Write Operand.L.
8.1		
8.2	Read PC:PB	Store as OpCode.
+X.1		

ORA (1D)		Cycles: 4	Size: 3
Absolute, X (Read)			
Cycle	R/W	Desc.	
-X.2	Read PC:PB	Store as OpCode.	
1.1		Inc. PC .	
1.2	Read PC:PB	Store as Pointer.	
2.1		Inc. PC .	
2.2	Read PC:PB	Store as Pointer.H.	
3.1		Inc. PC . Perform Orig = Pointer. Tmp = ui32(Pointer + X). Pointer = ui16(Tmp). If Orig.H == Pointer.H and the X flag is set, skip 2 half-cycles. Bank = DB + (Tmp >> 16).	
3.2			
4.1			
4.2	Read Pointer:Bank	Store as Operand. If M flag is set, skip the next cycle.	
5.1			
5.2	Read Pointer + 1:Bank (With Carry)	Store as Operand.H.	
6.1		Perform A = Operand. If M flag is set, set N based off (A .L & \$80) and Z based off A .L, otherwise set N based off (A .H & \$80) and Z based off A .	
6.2	Read PC:PB	Store as OpCode.	
+X.1			

ASL (1E)	Cycles: 7	Size: 3
Absolute, X (Read/Modiy/Write)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Store as Pointer.
2.1		Inc. PC .
2.2	Read PC:PB	Store as Pointer.H.
3.1		Inc. PC . Perform Tmp = ui32(Pointer + X). Pointer = ui16(Tmp). Bank = DB + (Tmp >> 16).
3.2		
4.1		
4.2	Read Pointer:Bank	Store as Operand. If M flag is set, skip the next cycle.
5.1		
5.2	Read Pointer + 1:Bank (With Carry)	Store as Operand.H.
6.1		If M flag is set, set C based off (Operand.L & \$80), perform Operand.L <= 1, and set N based off (Operand.L & \$80) and Z based off Operand.L, otherwise set C based off (Operand.H & \$80), perform Operand <= 1, and set N based off (Operand.H & \$80) and Z based off Operand.
6.2		If M flag is set, skip the next cycle.
7.1		
7.2	Write to Pointer + 1:Bank (With Carry)	Write Operand.H.
8.1		
8.2	Write to Pointer:Bank	Write Operand.L.
9.1		
9.2	Read PC:PB	Store as OpCode.
+X.1		

ORA (1F)	Cycles: 5	Size: 4
Absolute Long, X (Read)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Store as Pointer.
2.1		Inc. PC .
2.2	Read PC:PB	Store as Pointer.H.
3.1		Inc. PC .
3.2	Read PC:PB	Stores as Bank.
4.1		Inc. PC . Perform Tmp = ui32(Pointer + X). Pointer = ui16(Tmp). Bank += (Tmp >> 16).
4.2	Read Pointer:Bank	Store as Operand. If M flag is set, skip the next cycle.
5.1		
5.2	Read Pointer + 1:Bank (With Carry)	Store as Operand.H.
6.1		Perform A = Operand. If M flag is set, set N based off (A .L & \$80) and Z based off A .L, otherwise set N based off (A .H & \$80) and Z based off A .
6.2	Read PC:PB	Store as OpCode.
+X.1		

JSR (20)	Cycles: 6	Size: 3
Absolute		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Store as Address.
2.1		Inc. PC .
2.2	Read PC:PB	Store as Address.L.
3.1		Inc. PC .
3.2	Read PC:PB	Discard.
4.1		Dec. PC .
4.2	Write to (S+0)	Push PC .H onto stack.
5.1		
5.2	Write to (S-1)	Push PC .L onto stack.
6.1		Dec. S by 2. Perform PC = Address.
6.2	Read PC:PB	Store as OpCode.
+X.1		

AND (21)		Cycles: 6	Size: 2
Direct Indexed Indirect (d,x) (Read)			
Cycle	R/W	Desc.	
-X.2	Read PC:PB	Store as OpCode.	
1.1		Inc. PC .	
1.2	Read PC:PB	Store as Pointer.	
2.1		Inc. PC . Set Pointer to Pointer + X + D . Address = Pointer.	
2.2		If D.L is 0, skip the next cycle.	
3.1		Address.H = Pointer.H (fixes high byte of address).	
3.2			
4.1			
4.2	Read Address	Store as Pointer.	
5.1			
5.2	Read Address + 1	Store as Pointer.H.	
6.1			
6.2	Read Pointer: DB	Store as Operand. If M flag is set, skip the next cycle.	
7.1			
7.2	Read Pointer + 1: DB	Store as Operand.H.	
8.1		Perform A &= Operand. If M flag is set, set N based off (A.L & \$80) and Z based off A.L , otherwise set N based off (A.H & \$80) and Z based off A .	
8.2	Read PC:PB	Store as OpCode.	
+X.1			

JSL (22)	Cycles: 8	Size: 4
Absolute Long		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Store as Address.
2.1		Inc. PC .
2.2	Read PC:PB	Store as Address.L.
3.1		Inc. PC .
3.2	Write to (S +0)	Push PB onto stack.
4.1		
4.2	Read (S +0)	Discard.
5.1		
5.2	Read PC:PB	Stores as Bank.
6.1		
6.2	Write to (S -1)	Push PC .H onto stack.
7.1		
7.2	Write to (S -2)	Push PC .L onto stack.
8.1		Inc PC . Dec. S by 3. Copy Address to PC . Copy Bank to PB .
8.2	Read PC:PB	Store as OpCode.
+X.1		

AND (23)		Cycles: 4	Size: 2
Stack Relative (Read)			
Cycle	R/W	Desc.	
-X.2	Read PC:PB	Store as OpCode.	
1.1	Read PC:PB	Inc. PC .	
1.2		Store as Pointer.	
2.1		Inc. PC . Set Address to Pointer + S .	
2.2			
3.1			
3.2	Read Address	Store as Operand. If M flag is set, skip the next cycle.	
4.1	Read Address + 1		
4.2		Store as Operand.H.	
5.1	Read PC:PB	Perform A &= Operand. If M flag is set, set N based off (A .L & \$80) and Z based off A .L, otherwise set N based off (A .H & \$80) and Z based off A .	
5.2		Store as OpCode.	
+X.1			

BIT (24)	Cycles: 3	Size: 2
Direct (Read)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Store as Operand. If D.L is 0, skip the next cycle.
2.1		Inc. PC .
2.2		
3.1		Inc. PC if previous cycle was skipped. Set Address to Operand + D .
3.2	Read Address	Store as Operand. If M flag is set, skip the next cycle.
4.1		
4.2	Read Address + 1	Store as Operand.H.
5.1		If M flag is set, set Z based off (A.L & Operand.L), set N based off (Operand.L & \$80), and set V based off (Operand.L & \$40), otherwise set Z based off (A & Operand), set N based off (Operand.H & \$80), set V based off (Operand.H & \$40).
5.2	Read PC:PB	Store as OpCode.
+X.1		

AND (25)		Cycles: 3	Size: 2
Direct (Read)			
Cycle	R/W	Desc.	
-X.2	Read PC:PB	Store as OpCode.	
1.1		Inc. PC .	
1.2	Read PC:PB	Store as Operand. If D.L is 0, skip the next cycle.	
2.1		Inc. PC .	
2.2			
3.1		Inc. PC if previous cycle was skipped. Set Address to Operand + D .	
3.2	Read Address	Store as Operand. If M flag is set, skip the next cycle.	
4.1			
4.2	Read Address + 1	Store as Operand.H.	
5.1		Perform A &= Operand. If M flag is set, set N based off (A.L & \$80) and Z based off A.L , otherwise set N based off (A.H & \$80) and Z based off A .	
5.2	Read PC:PB	Store as OpCode.	
+X.1			

ROL (26)		Cycles: 5	Size: 2
Direct (Read/Modify/Write)			
Cycle	R/W	Desc.	
-X.2	Read PC:PB	Store as OpCode.	
1.1		Inc. PC .	
1.2	Read PC:PB	Store as Operand. If D.L is 0, skip the next cycle.	
2.1		Inc. PC .	
2.2			
3.1		Inc. PC if previous cycle was skipped. Set Address to Operand + D .	
3.2	Read Address	Store as Operand. If M flag is set, skip the next cycle.	
4.1			
4.2	Read Address + 1	Store as Operand.H.	
5.1		If M flag is set, set C based off (Operand.L & \$80), perform Operand.L = (Operand.L << 1) C , and set N based off (Operand.L & \$80) and Z based off Operand.L, otherwise set C based off (Operand.H & \$80), perform Operand = (Operand << 1) C , and set N based off (Operand.H & \$80) and Z based off Operand.	
5.2		If M flag is set, skip the next cycle.	
6.1			
6.2	Write to Address + 1	Write Operand.H.	
7.1			
7.2	Write to Address	Write Operand.L.	
8.1			
8.2	Read PC:PB	Store as OpCode.	
+X.1			

AND (27)		Cycles: 6	Size: 2
Direct Indirect Long (Read)			
Cycle	R/W	Desc.	
-X.2	Read PC:PB	Store as OpCode.	
1.1		Inc. PC .	
1.2	Read PC:PB	Store as Operand. If D.L is 0, skip the next cycle.	
2.1		Inc. PC .	
2.2			
3.1		Inc. PC if previous cycle was skipped. Set Address to Operand + D .	
3.2	Read Address	Store as Pointer.	
4.1			
4.2	Read Address + 1	Store as Pointer.H.	
5.1			
5.2	Read Address + 2	Store as Bank.	
6.1			
6.2	Read Pointer:Bank	Store as Operand. If M flag is set, skip the next cycle.	
7.1			
7.2	Read Pointer + 1:Bank (With Carry)	Store as Operand.H.	
8.1		Perform A &= Operand. If M flag is set, set N based off (A.L & \$80) and Z based off A.L , otherwise set N based off (A.H & \$80) and Z based off A .	
8.2	Read PC:PB	Store as OpCode.	
+X.1			

PLP (28)	Cycles: 4	Size: 1
Implied		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2		
2.1		
2.2		
3.1		Inc. S by 1.
3.2	Read S	Store as Operand.L.
4.1		P = Operand.L, set X.H and Y.H to 0 if X flag is set.
4.2	Read PC:PB	Store as OpCode.
+X.1		

AND (29)	Cycles: 3	Size: 2
Immediate		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Store as Operand. If M flag is set, skip the next cycle.
2.1		Inc. PC .
2.2	Read PC:PB	Store as Operand.H.
3.1		Inc. PC . Perform A &= Operand. If M flag is set, set N based off (A .L & \$80) and Z based off A .L, otherwise set N based off (A .H & \$80) and Z based off A .
3.2	Read PC:PB	Store as OpCode.
+X.1		

ROL (2A)	Cycles: 2	Size: 1
Implied		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1	Read PC:PB	Inc. PC .
1.2		Discard.
2.1		If M flag is set, set C based off (A.L & \$80), perform A.L = (A.L << 1) C , and set N based off (A.L & \$80) and Z based off A.L , otherwise set C based off (A.H & \$80), perform A = (A << 1) C , and set N based off (A.H & \$80) and Z based off A .
2.2		Store as OpCode.
+X.1		

PLD (2B)	Cycles: 5	Size: 1
Implied		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1	Read (S +1)	Inc. PC .
1.2		
2.1		
2.2		
3.1		
3.2		Store as Operand.L.
4.1		
4.2		Store as Operand.H.
5.1		Inc. S by 2. Set D to Operand. Set N based off (D .H & \$80). Set Z based off D .
5.2		Store as OpCode.
+X.1		

BIT (2C)	Cycles: 4	Size: 3
Absolute (Read)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Store as Address.
2.1		Inc. PC .
2.2	Read PC:PB	Store as Address.L.
3.1		Inc. PC .
3.2	Read Address: DB	Store as Operand. If M flag is set, skip the next cycle.
4.1		
4.2	Read Address + 1: DB	Store as Operand.H.
5.1		If M flag is set, set Z based off (A .L & Operand.L), set N based off (Operand.L & \$80), and set V based off (Operand.L & \$40), otherwise set Z based off (A & Operand), set N based off (Operand.H & \$80), and set V based off (Operand.H & \$40).
5.2	Read PC:PB	Store as OpCode.
+X.1		

AND (2D)		Cycles: 4	Size: 3
Absolute (Read)			
Cycle	R/W	Desc.	
-X.2	Read PC:PB	Store as OpCode.	
1.1		Inc. PC .	
1.2	Read PC:PB	Store as Address.	
2.1		Inc. PC .	
2.2	Read PC:PB	Store as Address.L.	
3.1		Inc. PC .	
3.2	Read Address: DB	Store as Operand. If M flag is set, skip the next cycle.	
4.1			
4.2	Read Address + 1: DB	Store as Operand.H.	
5.1		Perform A &= Operand. If M flag is set, set N based off (A .L & \$80) and Z based off A .L, otherwise set N based off (A .H & \$80) and Z based off A .	
5.2	Read PC:PB	Store as OpCode.	
+X.1			

ROL (2E)		Cycles: 6	Size: 3
Absolute (Read/Modify/Write)			
Cycle	R/W	Desc.	
-X.2	Read PC:PB	Store as OpCode.	
1.1		Inc. PC .	
1.2	Read PC:PB	Store as Address.	
2.1		Inc. PC .	
2.2	Read PC:PB	Store as Address.L.	
3.1		Inc. PC .	
3.2	Read Address: DB	Store as Operand. If M flag is set, skip the next cycle.	
4.1			
4.2	Read Address + 1: DB	Store as Operand.H.	
5.1		If M flag is set, set C based off (Operand.L & \$80), perform Operand.L = (Operand.L << 1) C , and set N based off (Operand.L & \$80) and Z based off Operand.L, otherwise set C based off (Operand.H & \$80), perform Operand = (Operand << 1) C , and set N based off (Operand.H & \$80) and Z based off Operand.	
5.2		If M flag is set, skip the next cycle.	
6.1			
6.2	Write to Address + 1: DB	Write Operand.H.	
7.1			
7.2	Write to Address: DB	Write Operand.L.	
8.1			
8.2	Read PC:PB	Store as OpCode.	
+X.1			

AND (2F)	Cycles: 5	Size: 4
Absolute Long (Read)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Store as Address.
2.1		Inc. PC .
2.2	Read PC:PB	Store as Address.L.
3.1		Inc. PC .
3.2	Read PC:PB	Stores as Bank.
4.1		Inc. PC .
4.2	Read Address:Bank	Store as Operand. If M flag is set, skip the next cycle.
5.1		
5.2	Read Address + 1:Bank (With Carry)	Store as Operand.H.
6.1		Perform A &= Operand. If M flag is set, set N based off (A .L & \$80) and Z based off A .L, otherwise set N based off (A .H & \$80) and Z based off A .
6.2	Read PC:PB	Store as OpCode.
+X.1		

BMI (30)	Cycles: 2	Size: 2
Relative		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC . Decide to branch if N is 1.
1.2	Read PC:PB	Store as Operand. Address = i16(i8(Operand.L)) + PC . If not branching, poll interrupts.
2.1		Inc. PC . If not branching, end (next half-cycle is 3.2).
2.2		Poll interrupts.
3.1		Set PC to Address.
3.2	Read PC:PB	Store as OpCode.
+X.1		

AND (31)		Cycles: 5	Size: 2
Direct Indirect Indexed (d),y (Read)			
Cycle	R/W	Desc.	
-X.2	Read PC:PB	Store as OpCode.	
1.1		Inc. PC .	
1.2	Read PC:PB	Store as Operand. If D.L is 0, skip the next cycle.	
2.1		Inc. PC .	
2.2			
3.1		Inc. PC if previous cycle was skipped. Set Address to Operand + D .	
3.2	Read Address	Store as Pointer.	
4.1			
4.2	Read Address + 1	Store as Pointer.H.	
5.1		Perform Orig = Pointer. Tmp = ui32(Pointer + Y). Pointer = ui16(Tmp). If Orig.H == Pointer.H and the X flag is set, skip 2 half-cycles. Bank = DB + (Tmp >> 16).	
5.2			
6.1			
6.2	Read Pointer:Bank	Store as Operand. If M flag is set, skip the next cycle.	
7.1			
7.2	Read Pointer + 1:Bank (With Carry)	Store as Operand.H.	
8.1		Perform A &= Operand. If M flag is set, set N based off (A.L & \$80) and Z based off A.L , otherwise set N based off (A.H & \$80) and Z based off A .	
8.2	Read PC:PB	Store as OpCode.	
+X.1			

AND (32)	Cycles: 5	Size: 2
Direct Indirect (Read)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Store as Operand. If D.L is 0, skip the next cycle.
2.1		Inc. PC .
2.2		
3.1		Inc. PC if previous cycle was skipped. Set Address to Operand + D .
3.2	Read Address	Store as Pointer.
4.1		
4.2	Read Address + 1	Store as Pointer.H.
5.1		
5.2	Read Pointer: DB	Store as Operand. If M flag is set, skip the next cycle.
6.1		
6.2	Read Pointer + 1: DB	Store as Operand.H.
7.1		Perform A &= Operand. If M flag is set, set N based off (A.L & \$80) and Z based off A.L , otherwise set N based off (A.H & \$80) and Z based off A .
7.2	Read PC:PB	Store as OpCode.
+X.1		

AND (33)		Cycles: 7	Size: 2
Stack Relative Indirect Indexed (Read)			
Cycle	R/W	Desc.	
-X.2	Read PC:PB	Store as OpCode.	
1.1	Read PC:PB	Inc. PC .	
1.2		Store as Pointer.	
2.1		Inc. PC . Set Address to Pointer + S .	
2.2			
3.1			
3.2	Read Address	Store as Pointer.	
4.1	Read Address + 1	Store as Pointer.H.	
4.2		Perform Tmp = ui32(Pointer + Y).	
5.1		Pointer = ui16(Tmp).	
		Bank = DB + (Tmp >> 16).	
5.2			
6.1	Read Pointer:Bank		
6.2		Store as Operand. If M flag is set, skip the next cycle.	
7.1	Read Pointer + 1:Bank (With Carry)		
7.2		Store as Operand.H.	
8.1	Read PC:PB	Perform A &= Operand. If M flag is set, set N based off (A .L & \$80) and Z based off A .L, otherwise set N based off (A .H & \$80) and Z based off A .	
8.2		Store as OpCode.	
+X.1			

BIT (34)	Cycles: 4	Size: 2
Direct, X (Read)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1	Read PC:PB	Inc. PC .
1.2		Store as Operand.
2.1		Inc. PC .
2.2		
3.1		Address = Operand + X + D . If D.L is 0, skip 2 half-cycles.
3.2		
4.1		
4.2	Read Address	Store as Operand. If M flag is set, skip the next cycle.
5.1		
5.2	Read Address + 1	Store as Operand.H.
6.1		If M flag is set, set Z based off (A.L & Operand.L), set N based off (Operand.L & \$80), and set V based off (Operand.L & \$40), otherwise set Z based off (A & Operand), set N based off (Operand.H & \$80), and set V based off (Operand.H & \$40).
6.2	Read PC:PB	Store as OpCode.
+X.1		

AND (35)		Cycles: 4	Size: 2
Direct, X (Read)			
Cycle	R/W	Desc.	
-X.2	Read PC:PB	Store as OpCode.	
1.1	Read PC:PB	Inc. PC .	
1.2		Store as Operand.	
2.1		Inc. PC .	
2.2			
3.1		Address = Operand + X + D . If D.L is 0, skip 2 half-cycles.	
3.2			
4.1			
4.2	Read Address	Store as Operand. If M flag is set, skip the next cycle.	
5.1	Read Address + 1		
5.2		Store as Operand.H.	
6.1	Read PC:PB	Perform A &= Operand. If M flag is set, set N based off (A.L & \$80) and Z based off A.L , otherwise set N based off (A.H & \$80) and Z based off A .	
6.2		Store as OpCode.	
+X.1			

ROL (36)	Cycles: 6	Size: 2
Direct Indexed Indirect (d,x) (Read/Modify/Write)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Store as Operand.
2.1		Inc. PC .
2.2		
3.1		Address = Operand + X + D . If D.L is 0, skip 2 half-cycles.
3.2		
4.1		
4.2	Read Address	Store as Operand. If M flag is set, skip the next cycle.
5.1		
5.2	Read Address + 1	Store as Operand.H.
6.1		If M flag is set, set C based off (Operand.L & \$80), perform Operand.L = (Operand.L << 1) C , and set N based off (Operand.L & \$80) and Z based off Operand.L, otherwise set C based off (Operand.H & \$80), perform Operand = (Operand << 1) C , and set N based off (Operand.H & \$80) and Z based off Operand.
6.2		If M flag is set, skip the next cycle.
7.1		
7.2	Write to Address + 1	Write Operand.H.
8.1		
8.2	Write to Address	Write Operand.L.
9.1		
9.2	Read PC:PB	Store as OpCode.
+X.1		

AND (37)		Cycles: 6	Size: 2
Direct Indirect Indexed Long [d],y (Read)			
Cycle	R/W	Desc.	
-X.2	Read PC:PB	Store as OpCode.	
1.1		Inc. PC .	
1.2	Read PC:PB	Store as Operand. If D.L is 0, skip the next cycle.	
2.1		Inc. PC .	
2.2			
3.1		Inc. PC if previous cycle was skipped. Set Address to Operand + D .	
3.2	Read Address	Store as Pointer.	
4.1			
4.2	Read Address + 1	Store as Pointer.H.	
5.1			
5.2	Read Address + 2	Store as Bank.	
6.1		Perform $\text{Tmp} = \text{ui32}(\text{Pointer} + \mathbf{Y})$.	
		Pointer = $\text{ui16}(\text{Tmp})$.	
		Bank += (Tmp >> 16).	
6.2	Read Pointer:Bank	Store as Operand. If M flag is set, skip the next cycle.	
7.1			
7.2	Read Pointer + 1:Bank (With Carry)	Store as Operand.H.	
8.1		Perform A &= Operand. If M flag is set, set N based off (A.L & \$80) and Z based off A.L , otherwise set N based off (A.H & \$80) and Z based off A .	
8.2	Read PC:PB	Store as OpCode.	
+X.1			

SEC (38)	Cycles: 2	Size: 1
Implied		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Discard.
2.1		Sets the C flag to 1.
2.2	Read PC:PB	Store as OpCode.
+X.1		

AND (39)		Cycles: 4	Size: 3
Absolute, Y (Read)			
Cycle	R/W	Desc.	
-X.2	Read PC:PB	Store as OpCode.	
1.1		Inc. PC .	
1.2	Read PC:PB	Store as Pointer.	
2.1		Inc. PC .	
2.2	Read PC:PB	Store as Pointer.H.	
3.1		Inc. PC . Perform Orig = Pointer. Tmp = ui32(Pointer + Y). Pointer = ui16(Tmp). If Orig.H == Pointer.H and the X flag is set, skip 2 half-cycles. Bank = DB + (Tmp >> 16).	
3.2			
4.1			
4.2	Read Pointer:Bank	Store as Operand. If M flag is set, skip the next cycle.	
5.1			
5.2	Read Pointer + 1:Bank (With Carry)	Store as Operand.H.	
6.1		Perform A &= Operand. If M flag is set, set N based off (A .L & \$80) and Z based off A .L, otherwise set N based off (A .H & \$80) and Z based off A .	
6.2	Read PC:PB	Store as OpCode.	
+X.1			

DEC (3A)	Cycles: 2	Size: 1
Implied		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1	Read PC:PB	Inc. PC .
1.2		Discard.
2.1		If M flag is set, perform A.L -= 1, set N based off (A.L & \$80), and set Z based off A.L , otherwise perform A -= 1, set N based off (A.H & \$80), and set Z based off A .
2.2		Store as OpCode.
+X.1		

TSC (3B)	Cycles: 2	Size: 1
Implied		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1	Read PC:PB	Inc. PC .
1.2		Discard.
2.1		Set A to S . Set N based off (A .H & \$80) and Z based off A .
2.2		Store as OpCode.
+X.1		

BIT (3C)	Cycles: 4	Size: 3
Absolute, X (Read)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Store as Pointer.
2.1		Inc. PC .
2.2	Read PC:PB	Store as Pointer.H.
3.1		Inc. PC . Perform Orig = Pointer. Tmp = ui32(Pointer + X). Pointer = ui16(Tmp). If Orig.H == Pointer.H and the X flag is set, skip 2 half-cycles. Bank = DB + (Tmp >> 16).
3.2		
4.1		
4.2	Read Pointer:Bank	Store as Operand. If M flag is set, skip the next cycle.
5.1		
5.2	Read Pointer + 1:Bank (With Carry)	Store as Operand.H.
6.1		If M flag is set, set Z based off (A .L & Operand.L), set N based off (Operand.L & \$80), and set V based off (Operand.L & \$40), otherwise set Z based off (A & Operand), set N based off (Operand.H & \$80), and set V based off (Operand.H & \$40).
6.2	Read PC:PB	Store as OpCode.
+X.1		

AND (3D)		Cycles: 4	Size: 3
Absolute, X (Read)			
Cycle	R/W	Desc.	
-X.2	Read PC:PB	Store as OpCode.	
1.1		Inc. PC .	
1.2	Read PC:PB	Store as Pointer.	
2.1		Inc. PC .	
2.2	Read PC:PB	Store as Pointer.H.	
3.1		Inc. PC . Perform Orig = Pointer. Tmp = ui32(Pointer + X). Pointer = ui16(Tmp). If Orig.H == Pointer.H and the X flag is set, skip 2 half-cycles. Bank = DB + (Tmp >> 16).	
3.2			
4.1			
4.2	Read Pointer:Bank	Store as Operand. If M flag is set, skip the next cycle.	
5.1			
5.2	Read Pointer + 1:Bank (With Carry)	Store as Operand.H.	
6.1		Perform A &= Operand. If M flag is set, set N based off (A .L & \$80) and Z based off A .L, otherwise set N based off (A .H & \$80) and Z based off A .	
6.2	Read PC:PB	Store as OpCode.	
+X.1			

ROL (3E)		Cycles: 7	Size: 3
Absolute, X (Read/Modiy/Write)			
Cycle	R/W	Desc.	
-X.2	Read PC:PB	Store as OpCode.	
1.1		Inc. PC .	
1.2	Read PC:PB	Store as Pointer.	
2.1		Inc. PC .	
2.2	Read PC:PB	Store as Pointer.H.	
3.1		Inc. PC . Perform $\text{Tmp} = \text{ui32}(\text{Pointer} + \text{X})$.	
		Pointer = $\text{ui16}(\text{Tmp})$.	
		Bank = $\text{DB} + (\text{Tmp} \gg 16)$.	
3.2			
4.1			
4.2	Read Pointer:Bank	Store as Operand. If M flag is set, skip the next cycle.	
5.1			
5.2	Read Pointer + 1:Bank (With Carry)	Store as Operand.H.	
6.1		If M flag is set, set C based off (Operand.L & \$80), perform $\text{Operand.L} = (\text{Operand.L} \ll 1) \mid \text{C}$, and set N based off (Operand.L & \$80) and Z based off Operand.L, otherwise set C based off (Operand.H & \$80), perform $\text{Operand} = (\text{Operand} \ll 1) \mid \text{C}$, and set N based off (Operand.H & \$80) and Z based off Operand.	
6.2		If M flag is set, skip the next cycle.	
7.1			
7.2	Write to Pointer + 1:Bank (With Carry)	Write Operand.H.	
8.1			
8.2	Write to Pointer:Bank	Write Operand.L.	
9.1			
9.2	Read PC:PB	Store as OpCode.	
+X.1			

AND (3F)		Cycles: 5	Size: 4
Absolute Long, X (Read)			
Cycle	R/W	Desc.	
-X.2	Read PC:PB	Store as OpCode.	
1.1		Inc. PC .	
1.2	Read PC:PB	Store as Pointer.	
2.1		Inc. PC .	
2.2	Read PC:PB	Store as Pointer.H.	
3.1		Inc. PC .	
3.2	Read PC:PB	Stores as Bank.	
4.1		Inc. PC . Perform $\text{Tmp} = \text{ui32}(\text{Pointer} + \text{X})$. Pointer = $\text{ui16}(\text{Tmp})$. Bank += (Tmp >> 16).	
4.2	Read Pointer:Bank	Store as Operand. If M flag is set, skip the next cycle.	
5.1			
5.2	Read Pointer + 1:Bank (With Carry)	Store as Operand.H.	
6.1		Perform A &= Operand. If M flag is set, set N based off (A .L & \$80) and Z based off A .L, otherwise set N based off (A .H & \$80) and Z based off A .	
6.2	Read PC:PB	Store as OpCode.	
+X.1			

RTI (40)	Cycles: 7	Size: 1
Implied		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2		
2.1		
2.2		
3.1		Inc. S by 1.
3.2	Read (S +0)	Store as Operand.L.
4.1		Set P to Operand.
4.2	Read (S +1)	Store as Operand.L.
5.1		
5.2	Read (S +2)	Store as Operand.H.
6.1		
6.2	Read (S +3)	Store as PB .
7.1		Inc. S by 3. Set PB to Bank. Set PC to Operand. If X flag is set, set X to X.L and Y to Y.L .
7.2	Read PC:PB	Store as OpCode.
+X.1		

EOR (41)	Cycles: 6	Size: 2
Direct Indexed Indirect (d,x) (Read)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Store as Pointer.
2.1		Inc. PC . Set Pointer to Pointer + X + D . Address = Pointer.
2.2		If D.L is 0, skip the next cycle.
3.1		Address.H = Pointer.H (fixes high byte of address).
3.2		
4.1		
4.2	Read Address	Store as Pointer.
5.1		
5.2	Read Address + 1	Store as Pointer.H.
6.1		
6.2	Read Pointer: DB	Store as Operand. If M flag is set, skip the next cycle.
7.1		
7.2	Read Pointer + 1: DB	Store as Operand.H.
8.1		Perform A ^= Operand. If M flag is set, set N based off (A.L & \$80) and Z based off A.L , otherwise set N based off (A.H & \$80) and Z based off A .
8.2	Read PC:PB	Store as OpCode.
+X.1		

WDM (42) Cycles: 2		Size: 2
Immediate		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Store as Operand.
2.1		Inc. PC .
2.2	Read PC:PB	Store as OpCode.
+X.1		

EOR (43)		Cycles: 4	Size: 2
Stack Relative (Read)			
Cycle	R/W	Desc.	
-X.2	Read PC:PB	Store as OpCode.	
1.1	Read PC:PB	Inc. PC .	
1.2		Store as Pointer.	
2.1		Inc. PC . Set Address to Pointer + S .	
2.2			
3.1			
3.2	Read Address	Store as Operand. If M flag is set, skip the next cycle.	
4.1	Read Address + 1		
4.2		Store as Operand.H.	
5.1	Read PC:PB	Perform A ^= Operand. If M flag is set, set N based off (A .L & \$80) and Z based off A .L, otherwise set N based off (A .H & \$80) and Z based off A .	
5.2		Store as OpCode.	
+X.1			

MVP (44)	Cycles: 7	Size: 3
Block Move (Positive)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Store as Operand.
2.1		Inc. PC . Set DB to Operand.L.
2.2	Read PC:PB	Stores as Bank.
3.1		Inc. PC .
3.2	Read X:Bank	Store as Operand.L.
4.1		
4.2	Write Y:DB	Write Operand.L.
5.1		Perform A -= 1. If X flag is set, perform X.L -= 1 and Y.L -= 1, otherwise perform X -= 1 and Y -= 1. If A is not \$FFFF, perform PC -= 3.
5.2		
6.1		
6.2		
7.1		
7.2	Read PC:PB	Store as OpCode.
+X.1		

EOR (45)		Cycles: 3	Size: 2
Direct (Read)			
Cycle	R/W	Desc.	
-X.2	Read PC:PB	Store as OpCode.	
1.1		Inc. PC .	
1.2	Read PC:PB	Store as Operand. If D.L is 0, skip the next cycle.	
2.1		Inc. PC .	
2.2			
3.1		Inc. PC if previous cycle was skipped. Set Address to Operand + D .	
3.2	Read Address	Store as Operand. If M flag is set, skip the next cycle.	
4.1			
4.2	Read Address + 1	Store as Operand.H.	
5.1		Perform A ^= Operand. If M flag is set, set N based off (A.L & \$80) and Z based off A.L , otherwise set N based off (A.H & \$80) and Z based off A .	
5.2	Read PC:PB	Store as OpCode.	
+X.1			

LSR (46)	Cycles: 5	Size: 2
Direct (Read/Modify/Write)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Store as Operand. If D.L is 0, skip the next cycle.
2.1		Inc. PC .
2.2		
3.1		Inc. PC if previous cycle was skipped. Set Address to Operand + D .
3.2	Read Address	Store as Operand. If M flag is set, skip the next cycle.
4.1		
4.2	Read Address + 1	Store as Operand.H.
5.1		If M flag is set, set C based off (Operand.L & \$01), perform Operand.L >>= 1, and set N based off (Operand.L & \$80) and Z based off Operand.L, otherwise set C based off (Operand.L & \$01), perform Operand >>= 1, and set N based off (Operand.H & \$80) and Z based off Operand.
5.2		If M flag is set, skip the next cycle.
6.1		
6.2	Write to Address + 1	Write Operand.H.
7.1		
7.2	Write to Address	Write Operand.L.
8.1		
8.2	Read PC:PB	Store as OpCode.
+X.1		

EOR (47)	Cycles: 6	Size: 2
Direct Indirect Long (Read)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Store as Operand. If D.L is 0, skip the next cycle.
2.1		Inc. PC .
2.2		
3.1		Inc. PC if previous cycle was skipped. Set Address to Operand + D .
3.2	Read Address	Store as Pointer.
4.1		
4.2	Read Address + 1	Store as Pointer.H.
5.1		
5.2	Read Address + 2	Store as Bank.
6.1		
6.2	Read Pointer:Bank	Store as Operand. If M flag is set, skip the next cycle.
7.1		
7.2	Read Pointer + 1:Bank (With Carry)	Store as Operand.H.
8.1		Perform A ^= Operand. If M flag is set, set N based off (A.L & \$80) and Z based off A.L , otherwise set N based off (A.H & \$80) and Z based off A .
8.2	Read PC:PB	Store as OpCode.
+X.1		

PHA (48)	Cycles: 3	Size: 1
Implied		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2		If M flag is set, skip the next cycle.
2.1		
2.2	Write to (S +0)	Push A.H onto stack.
3.1		Dec. S by 1.
3.2	Write to (S +0)	Push A.L onto stack.
4.1		Dec. S by 1.
4.2	Read PC:PB	Store as OpCode.
+X.1		

EOR (49)	Cycles: 3	Size: 2
Immediate		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Store as Operand. If M flag is set, skip the next cycle.
2.1		Inc. PC .
2.2	Read PC:PB	Store as Operand.H.
3.1		Inc. PC . Perform A ^= Operand . If M flag is set, set N based off (A.L & \$80) and Z based off A.L , otherwise set N based off (A.H & \$80) and Z based off A .
3.2	Read PC:PB	Store as OpCode.
+X.1		

LSR (4A)	Cycles: 2	Size: 1
Implied		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1	Read PC:PB	Inc. PC .
1.2		Discard.
2.1		If M flag is set, set C based off (A.L & \$01), perform A.L >>= 1, and set N based off (A.L & \$80) and Z based off A.L ,
2.2		otherwise set C based off (A.L & \$01), perform A >>= 1, and set N based off (A.H & \$80) and Z based off A .
+X.1	Read PC:PB	Store as OpCode.

PHK (4B)		Cycles: 3	Size: 1
Implied			
Cycle	R/W		Desc.
-X.2	Read PC:PB		Store as OpCode.
1.1			Inc. PC .
1.2			
2.1			
2.2	Write to (S +0)		Push PB onto stack.
3.1			Dec. S by 1.
3.2	Read PC:PB		Store as OpCode.
+X.1			

JMP (4C)	Cycles: 3	Size: 3
Absolute		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Store as Address.
2.1		Inc. PC .
2.2	Read PC:PB	Store as Address.L.
3.1		Set PC to Address.
3.2	Read PC:PB	Store as OpCode.
+X.1		

EOR (4D)		Cycles: 4	Size: 3
Absolute (Read)			
Cycle	R/W	Desc.	
-X.2	Read PC:PB	Store as OpCode.	
1.1		Inc. PC .	
1.2	Read PC:PB	Store as Address.	
2.1		Inc. PC .	
2.2	Read PC:PB	Store as Address.L.	
3.1		Inc. PC .	
3.2	Read Address: DB	Store as Operand. If M flag is set, skip the next cycle.	
4.1			
4.2	Read Address + 1: DB	Store as Operand.H.	
5.1		Perform A ^= Operand. If M flag is set, set N based off (A .L & \$80) and Z based off A .L, otherwise set N based off (A .H & \$80) and Z based off A .	
5.2	Read PC:PB	Store as OpCode.	
+X.1			

LSR (4E)	Cycles: 6	Size: 3
Absolute (Read/Modify/Write)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Store as Address.
2.1		Inc. PC .
2.2	Read PC:PB	Store as Address.L.
3.1		Inc. PC .
3.2	Read Address: DB	Store as Operand. If M flag is set, skip the next cycle.
4.1		
4.2	Read Address + 1: DB	Store as Operand.H.
5.1		If M flag is set, set C based off (Operand.L & \$01), perform Operand.L >>= 1, and set N based off (Operand.L & \$80) and Z based off Operand.L, otherwise set C based off (Operand.L & \$01), perform Operand >>= 1, and set N based off (Operand.H & \$80) and Z based off Operand.
5.2		If M flag is set, skip the next cycle.
6.1		
6.2	Write to Address + 1: DB	Write Operand.H.
7.1		
7.2	Write to Address: DB	Write Operand.L.
8.1		
8.2	Read PC:PB	Store as OpCode.
+X.1		

EOR (4F)	Cycles: 5	Size: 4
Absolute Long (Read)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Store as Address.
2.1		Inc. PC .
2.2	Read PC:PB	Store as Address.L.
3.1		Inc. PC .
3.2	Read PC:PB	Stores as Bank.
4.1		Inc. PC .
4.2	Read Address:Bank	Store as Operand. If M flag is set, skip the next cycle.
5.1		
5.2	Read Address + 1:Bank (With Carry)	Store as Operand.H.
6.1		Perform A ^= Operand. If M flag is set, set N based off (A .L & \$80) and Z based off A .L, otherwise set N based off (A .H & \$80) and Z based off A .
6.2	Read PC:PB	Store as OpCode.
+X.1		

BVC (50)	Cycles: 2	Size: 2
Relative		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC . Decide to branch if V is 0.
1.2	Read PC:PB	Store as Operand. Address = i16(i8(Operand.L)) + PC . If not branching, poll interrupts.
2.1		Inc. PC . If not branching, end (next half-cycle is 3.2).
2.2		Poll interrupts.
3.1		Set PC to Address.
3.2	Read PC:PB	Store as OpCode.
+X.1		

EOR (51)		Cycles: 5	Size: 2
Direct Indirect Indexed (d),y (Read)			
Cycle	R/W	Desc.	
-X.2	Read PC:PB	Store as OpCode.	
1.1		Inc. PC .	
1.2	Read PC:PB	Store as Operand. If D.L is 0, skip the next cycle.	
2.1		Inc. PC .	
2.2			
3.1		Inc. PC if previous cycle was skipped. Set Address to Operand + D .	
3.2	Read Address	Store as Pointer.	
4.1			
4.2	Read Address + 1	Store as Pointer.H.	
5.1		Perform Orig = Pointer. Tmp = ui32(Pointer + Y). Pointer = ui16(Tmp). If Orig.H == Pointer.H and the X flag is set, skip 2 half-cycles. Bank = DB + (Tmp >> 16).	
5.2			
6.1			
6.2	Read Pointer:Bank	Store as Operand. If M flag is set, skip the next cycle.	
7.1			
7.2	Read Pointer + 1:Bank (With Carry)	Store as Operand.H.	
8.1		Perform A ^= Operand. If M flag is set, set N based off (A.L & \$80) and Z based off A.L , otherwise set N based off (A.H & \$80) and Z based off A .	
8.2	Read PC:PB	Store as OpCode.	
+X.1			

EOR (52)	Cycles: 5	Size: 2
Direct Indirect (Read)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Store as Operand. If D.L is 0, skip the next cycle.
2.1		Inc. PC .
2.2		
3.1		Inc. PC if previous cycle was skipped. Set Address to Operand + D .
3.2	Read Address	Store as Pointer.
4.1		
4.2	Read Address + 1	Store as Pointer.H.
5.1		
5.2	Read Pointer: DB	Store as Operand. If M flag is set, skip the next cycle.
6.1		
6.2	Read Pointer + 1: DB	Store as Operand.H.
7.1		Perform A ^= Operand. If M flag is set, set N based off (A.L & \$80) and Z based off A.L , otherwise set N based off (A.H & \$80) and Z based off A .
7.2	Read PC:PB	Store as OpCode.
+X.1		

EOR (53)		Cycles: 7	Size: 2
Stack Relative Indirect Indexed (Read)			
Cycle	R/W	Desc.	
-X.2	Read PC:PB	Store as OpCode.	
1.1	Read PC:PB	Inc. PC .	
1.2		Store as Pointer.	
2.1		Inc. PC . Set Address to Pointer + S .	
2.2			
3.1			
3.2	Read Address	Store as Pointer.	
4.1	Read Address + 1	Store as Pointer.H.	
4.2		Perform Tmp = ui32(Pointer + Y).	
5.1		Pointer = ui16(Tmp).	
		Bank = DB + (Tmp >> 16).	
5.2			
6.1	Read Pointer:Bank		
6.2		Store as Operand. If M flag is set, skip the next cycle.	
7.1	Read Pointer + 1:Bank (With Carry)		
7.2		Store as Operand.H.	
8.1	Read PC:PB	Perform A ^= Operand. If M flag is set, set N based off (A .L & \$80) and Z based off A .L, otherwise set N based off (A .H & \$80) and Z based off A .	
8.2		Store as OpCode.	
+X.1			

MVN (54)	Cycles: 7	Size: 3
Block Move (Negative)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Store as Operand.
2.1		Inc. PC . Set DB to Operand.L.
2.2	Read PC:PB	Stores as Bank.
3.1		Inc. PC .
3.2	Read X:Bank	Store as Operand.L.
4.1		
4.2	Write Y:DB	Write Operand.L.
5.1		Perform A -= 1. If X flag is set, perform X.L += 1 and Y.L += 1, otherwise perform X += 1 and Y += 1. If A is not \$FFFF, perform PC -= 3.
5.2		
6.1		
6.2		
7.1		
7.2	Read PC:PB	Store as OpCode.
+X.1		

EOR (55)	Cycles: 4	Size: 2
Direct, X (Read)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Store as Operand.
2.1		Inc. PC .
2.2		
3.1		Address = Operand + X + D . If D.L is 0, skip 2 half-cycles.
3.2		
4.1		
4.2	Read Address	Store as Operand. If M flag is set, skip the next cycle.
5.1		
5.2	Read Address + 1	Store as Operand.H.
6.1		Perform A ^= Operand. If M flag is set, set N based off (A.L & \$80) and Z based off A.L , otherwise set N based off (A.H & \$80) and Z based off A .
6.2	Read PC:PB	Store as OpCode.
+X.1		

LSR (56)	Cycles: 6	Size: 2
Direct, X (Read/Modify/Write)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Store as Operand.
2.1		Inc. PC .
2.2		
3.1		Address = Operand + X + D . If D.L is 0, skip 2 half-cycles.
3.2		
4.1		
4.2	Read Address	Store as Operand. If M flag is set, skip the next cycle.
5.1		
5.2	Read Address + 1	Store as Operand.H.
6.1		If M flag is set, set C based off (Operand.L & \$01), perform Operand.L >>= 1, and set N based off (Operand.L & \$80) and Z based off Operand.L, otherwise set C based off (Operand.L & \$01), perform Operand >>= 1, and set N based off (Operand.H & \$80) and Z based off Operand.
6.2		If M flag is set, skip the next cycle.
7.1		
7.2	Write to Address + 1	Write Operand.H.
8.1		
8.2	Write to Address	Write Operand.L.
9.1		
9.2	Read PC:PB	Store as OpCode.
+X.1		

EOR (57)		Cycles: 6	Size: 2
Direct Indirect Indexed Long			
Cycle	R/W	Desc.	
-X.2	Read PC:PB	Store as OpCode.	
1.1		Inc. PC .	
1.2	Read PC:PB	Store as Operand. If D.L is 0, skip the next cycle.	
2.1		Inc. PC .	
2.2			
3.1		Inc. PC if previous cycle was skipped. Set Address to Operand + D .	
3.2	Read Address	Store as Pointer.	
4.1			
4.2	Read Address + 1	Store as Pointer.H.	
5.1			
5.2	Read Address + 2	Store as Bank.	
6.1		Perform $\text{Tmp} = \text{ui32}(\text{Pointer} + \text{Y})$.	
		Pointer = $\text{ui16}(\text{Tmp})$.	
		Bank += (Tmp >> 16).	
6.2	Read Pointer:Bank	Store as Operand. If M flag is set, skip the next cycle.	
7.1			
7.2	Read Pointer + 1:Bank (With Carry)	Store as Operand.H.	
8.1		Perform $\text{A} \wedge = \text{Operand}$. If M flag is set, set N based off (A.L & \$80) and Z based off A.L , otherwise set N based off (A.H & \$80) and Z based off A .	
8.2	Read PC:PB	Store as OpCode.	
+X.1			

CLI (58)	Cycles: 2	Size: 1
Implied		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Discard.
2.1		Sets the I flag to 0.
2.2	Read PC:PB	Store as OpCode.
+X.1		

EOR (59)		Cycles: 4	Size: 3
Absolute, Y (Read)			
Cycle	R/W	Desc.	
-X.2	Read PC:PB	Store as OpCode.	
1.1		Inc. PC .	
1.2	Read PC:PB	Store as Pointer.	
2.1		Inc. PC .	
2.2	Read PC:PB	Store as Pointer.H.	
3.1		Inc. PC . Perform Orig = Pointer. Tmp = ui32(Pointer + Y). Pointer = ui16(Tmp). If Orig.H == Pointer.H and the X flag is set, skip 2 half-cycles. Bank = DB + (Tmp >> 16).	
3.2			
4.1			
4.2	Read Pointer:Bank	Store as Operand. If M flag is set, skip the next cycle.	
5.1			
5.2	Read Pointer + 1:Bank (With Carry)	Store as Operand.H.	
6.1		Perform A ^= Operand. If M flag is set, set N based off (A .L & \$80) and Z based off A .L, otherwise set N based off (A .H & \$80) and Z based off A .	
6.2	Read PC:PB	Store as OpCode.	
+X.1			

PHY (5A)	Cycles: 3	Size: 1
Implied		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2		If X flag is set, skip the next cycle.
2.1		
2.2	Write to (S +0)	Push Y .H onto stack.
3.1		Dec. S by 1.
3.2	Write to (S +0)	Push Y .L onto stack.
4.1		Dec. S by 1.
4.2	Read PC:PB	Store as OpCode.
+X.1		

TCD (5B)	Cycles: 2	Size: 1
Implied		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1	Read PC:PB	Inc. PC .
1.2		Discard.
2.1		Set D to A . Set N based off (D .H & \$80) and Z based off D .
2.2		Store as OpCode.
+X.1		

JML (5C)	Cycles: 4	Size: 4
Absolute Long		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Store as Address.
2.1		Inc. PC .
2.2	Read PC:PB	Store as Address.H.
3.1		Inc. PC .
3.2	Read PC:PB	Stores as Bank.
4.1		Copy Address to PC . Copy Bank to PB .
4.2	Read PC:PB	Store as OpCode.
+X.1		

EOR (5D)		Cycles: 4	Size: 3
Absolute, X (Read)			
Cycle	R/W	Desc.	
-X.2	Read PC:PB	Store as OpCode.	
1.1		Inc. PC .	
1.2	Read PC:PB	Store as Pointer.	
2.1		Inc. PC .	
2.2	Read PC:PB	Store as Pointer.H.	
3.1		Inc. PC . Perform Orig = Pointer.	
		Tmp = ui32(Pointer + X).	
		Pointer = ui16(Tmp).	
		If Orig.H == Pointer.H and the X flag is set, skip 2 half-cycles.	
		Bank = DB + (Tmp >> 16).	
3.2			
4.1			
4.2	Read Pointer:Bank	Store as Operand. If M flag is set, skip the next cycle.	
5.1			
5.2	Read Pointer + 1:Bank (With Carry)	Store as Operand.H.	
6.1		Perform A ^= Operand. If M flag is set, set N based off (A .L & \$80) and Z based off A .L, otherwise set N based off (A .H & \$80) and Z based off A .	
6.2	Read PC:PB	Store as OpCode.	
+X.1			

LSR (5E)	Cycles: 7	Size: 3
Absolute, X (Read/Modiy/Write)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Store as Pointer.
2.1		Inc. PC .
2.2	Read PC:PB	Store as Pointer.H.
3.1		Inc. PC . Perform $\text{Tmp} = \text{ui32}(\text{Pointer} + \text{X})$. Pointer = $\text{ui16}(\text{Tmp})$. Bank = $\text{DB} + (\text{Tmp} \gg 16)$.
3.2		
4.1		
4.2	Read Pointer:Bank	Store as Operand. If M flag is set, skip the next cycle.
5.1		
5.2	Read Pointer + 1:Bank (With Carry)	Store as Operand.H.
6.1		If M flag is set, set C based off (Operand.L & \$01), perform $\text{Operand.L} \gg= 1$, and set N based off (Operand.L & \$80) and Z based off Operand.L, otherwise set C based off (Operand.L & \$01), perform $\text{Operand} \gg= 1$, and set N based off (Operand.H & \$80) and Z based off Operand.
6.2		If M flag is set, skip the next cycle.
7.1		
7.2	Write to Pointer + 1:Bank (With Carry)	Write Operand.H.
8.1		
8.2	Write to Pointer:Bank	Write Operand.L.
9.1		
9.2	Read PC:PB	Store as OpCode.
+X.1		

EOR (5F)		Cycles: 5	Size: 4
Absolute Long, X (Read)			
Cycle	R/W	Desc.	
-X.2	Read PC:PB	Store as OpCode.	
1.1		Inc. PC .	
1.2	Read PC:PB	Store as Pointer.	
2.1		Inc. PC .	
2.2	Read PC:PB	Store as Pointer.H.	
3.1		Inc. PC .	
3.2	Read PC:PB	Stores as Bank.	
4.1		Inc. PC . Perform $\text{Tmp} = \text{ui32}(\text{Pointer} + \text{X})$.	
		Pointer = $\text{ui16}(\text{Tmp})$.	
		Bank += (Tmp >> 16).	
4.2	Read Pointer:Bank	Store as Operand. If M flag is set, skip the next cycle.	
5.1			
5.2	Read Pointer + 1:Bank (With Carry)	Store as Operand.H.	
6.1		Perform $\text{A} \wedge = \text{Operand}$. If M flag is set, set N based off (A .L & \$80) and Z based off A .L, otherwise set N based off (A .H & \$80) and Z based off A .	
6.2	Read PC:PB	Store as OpCode.	
+X.1			

RTS (60)	Cycles: 6	Size: 1
Implied		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1	Read (S +0) Read (S +1)	Inc. PC .
1.2		
2.1		
2.2		
3.1		Inc. S by 1.
3.2		Store as Operand.L.
4.1		
4.2		Store as Operand.H.
5.1		
5.2		
6.1	Read PC:PB	Inc. S by 1. Set PC to (Operand + 1).
6.2		Store as OpCode.
+X.1		

ADC (61) Cycles: 6		Size: 2
Direct Indexed Indirect (d,x) (Read)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1	Read PC:PB	Inc. PC .
1.2		Store as Pointer.
2.1		Inc. PC . Set Pointer to Pointer + X + D . Address = Pointer.
2.2		If D.L is 0, skip the next cycle.
3.1		Address.H = Pointer.H (fixes high byte of address).
3.2		
4.1	Read Address	Store as Pointer.
4.2		
5.1	Read Address + 1	Store as Pointer.H.
5.2		
6.1		
6.2	Read Pointer: DB	Store as Operand. If M flag is set, skip the next cycle.
7.1	Read Pointer + 1: DB	Store as Operand.H.
7.2		
8.1		If M flag == 0 (8-bit): If D flag == 0: Result = ui16(A.L) + ui16(Operand) + C. V flag = (~ui16(A.L) ^ ui16(Operand)) & (ui16(A.L) ^ ui8(Result)) & \$80 != 0. C flag = Result > \$FF. Result = ui8(Result). Z flag = Result == \$00. N flag = (Result & \$80) != 0. If D flag == 1: Lo = ui16(A.L & 0x0F) + ui16(Operand & 0x0F) + C; if (Lo > 9) Lo += 6. CarryToHi = Lo > \$0F ? 1 : 0. HiSum = ui16(A.L >> 4) + ui16(Operand >> 4) + CarryToHi. V flag = (~((A.L >> 4) ^ (Operand >> 4)) & ((A.L >> 4) ^ HiSum) & \$08) != 0. if (HiSum > 9) HiSum += 6. C flag = HiSum > \$0F. Result = ui8(((HiSum & 0x0F) << 4) (Lo & \$0F)). Z flag = Result == \$00. N flag = (Result & \$80) != 0. If M flag == 0 (16-bit): If D flag == 0: Result = ui32(A) + ui32(Operand) + C. V flag = (~ui32(A) ^ ui32(Operand)) & (ui32(A) ^ ui16(Result)) & \$8000 != 0. C flag = Result > \$FFFF. Result = ui16(Result). Z flag = Result == \$0000. N flag = (Result & \$8000) != 0. If D flag == 1: Result = ui16(0). Carry = ui16(C). for i = 0, 4, 8, 12: DigitSum = ui16((A >> i) & \$0F) + ui16((Operand >> i) & \$0F) + Carry. if (i == 12) V flag = (~(A >> 12) ^ (Operand >> 12)) & ((A >> 12) ^ DigitSum) & \$08 != 0. if (DigitSum > 9) DigitSum += 6. Carry = (DigitSum > \$0F) ? 1 : 0. Result = ui16((DigitSum & \$0F) << i). C flag = Carry != 0. Z flag = Result != \$0000. N flag = (Result & \$8000) != 0.
8.2		Store as OpCode.
+X.1		

PER (62)	Cycles: 6	Size: 3
Implied		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Store as Operand.
2.1		Inc. PC .
2.2	Read PC:PB	Store as Operand.H.
3.1		Inc. PC . Set Operand to Operand + PC .
3.2		
4.1		
4.2	Write to (S +0)	Push Operand.H onto stack.
5.1		
5.2	Write to (S -1)	Push Operand.L onto stack.
6.1		Dec. S by 2.
6.2	Read PC:PB	Store as OpCode.
+X.1		

ADC (63) Cycles: 4		Size: 2
Stack Relative (Read)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1	Read PC:PB	Inc. PC .
1.2		Store as Pointer.
2.1		Inc. PC . Set Address to Pointer + S .
2.2		
3.1		
3.2	Read Address	Store as Operand. If M flag is set, skip the next cycle.
4.1	Read Address + 1	
4.2		Store as Operand.H.
5.1		If M flag == 0 (8-bit): If D flag == 0: Result = ui16(A.L) + ui16(Operand) + C. V flag = (~ui16(A.L) ^ ui16(Operand)) & (ui16(A.L) ^ ui8(Result)) & \$80) != 0. C flag = Result > \$FF. Result = ui8(Result). Z flag = Result == \$00. N flag = (Result & \$80) != 0. If D flag == 1: Lo = ui16(A.L & 0x0F) + ui16(Operand & 0x0F) + C; if (Lo > 9) Lo += 6. CarryToHi = Lo > \$0F ? 1 : 0. HiSum = ui16(A.L >> 4) + ui16(Operand >> 4) + CarryToHi. V flag = (~((A.L >> 4) ^ (Operand >> 4)) & ((A.L >> 4) ^ HiSum) & \$08) != 0. if (HiSum > 9) HiSum += 6. C flag = HiSum > \$0F. Result = ui8(((HiSum & 0x0F) << 4) (Lo & \$0F)). Z flag = Result == \$00. N flag = (Result & \$80) != 0. If M flag == 0 (16-bit): If D flag == 0: Result = ui32(A) + ui32(Operand) + C. V flag = (~ui32(A) ^ ui32(Operand)) & (ui32(A) ^ ui16(Result)) & \$8000) != 0). C flag = Result > \$FFFF. Result = ui16(Result). Z flag = Result == \$0000. N flag = (Result & \$8000) != 0. If D flag == 1: Result = ui16(0). Carry = ui16(C). for i = 0, 4, 8, 12: DigitSum = ui16((A >> i) & \$0F) + ui16((Operand >> i) & \$0F) + Carry. if (i == 12) V flag = (~(A >> 12) ^ (Operand >> 12)) & ((A >> 12) ^ DigitSum) & \$08) != 0. if (DigitSum > 9) DigitSum += 6. Carry = (DigitSum > \$0F) ? 1 : 0. Result = ui16((DigitSum & \$0F) << i). C flag = Carry != 0. Z flag = Result != \$0000. N flag = (Result & \$8000) != 0.
5.2		Read PC:PB
		Store as OpCode.
+X.1		

STZ (64)	Cycles: 3	Size: 2
Direct (Write)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Store as Operand. If D.L is 0, skip the next cycle.
2.1		Inc. PC .
2.2		
3.1		Inc. PC if previous cycle was skipped. Set Address to Operand + D .
3.2	Write to Address	Write 0. If M flag is set, skip the next cycle.
4.1		
4.2	Write to Address + 1	Write 0.
5.1		
5.2	Read PC:PB	Store as OpCode.
+X.1		

ADC (65) Cycles: 3		Size: 2
Direct (Read)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Store as Operand. If D.L is 0, skip the next cycle.
2.1		Inc. PC .
2.2		
3.1		Inc. PC if previous cycle was skipped. Set Address to Operand + D .
3.2	Read Address	Store as Operand. If M flag is set, skip the next cycle.
4.1		
4.2	Read Address + 1	Store as Operand.H.
5.1		If M flag == 0 (8-bit): If D flag == 0: Result = ui16(A.L) + ui16(Operand) + C. V flag = (~ui16(A.L) ^ ui16(Operand)) & (ui16(A.L) ^ ui8(Result)) & \$80 != 0. C flag = Result > \$FF. Result = ui8(Result). Z flag = Result == \$00. N flag = (Result & \$80) != 0. If D flag == 1: Lo = ui16(A.L & 0x0F) + ui16(Operand & 0x0F) + C; if (Lo > 9) Lo += 6. CarryToHi = Lo > \$0F ? 1 : 0. HiSum = ui16(A.L >> 4) + ui16(Operand >> 4) + CarryToHi. V flag = (~((A.L >> 4) ^ (Operand >> 4)) & ((A.L >> 4) ^ HiSum) & \$08) != 0. if (HiSum > 9) HiSum += 6. C flag = HiSum > \$0F. Result = ui8(((HiSum & 0x0F) << 4) (Lo & \$0F)). Z flag = Result == \$00. N flag = (Result & \$80) != 0. If M flag == 0 (16-bit): If D flag == 0: Result = ui32(A) + ui32(Operand) + C. V flag = (~ui32(A) ^ ui32(Operand)) & (ui32(A) ^ ui16(Result)) & \$8000 != 0. C flag = Result > \$FFFF. Result = ui16(Result). Z flag = Result == \$0000. N flag = (Result & \$8000) != 0. If D flag == 1: Result = ui16(0). Carry = ui16(C). for i = 0, 4, 8, 12: DigitSum = ui16((A >> i) & \$0F) + ui16((Operand >> i) & \$0F) + Carry. if (i == 12) V flag = (~(A >> 12) ^ (Operand >> 12)) & ((A >> 12) ^ DigitSum) & \$08 != 0. if (DigitSum > 9) DigitSum += 6. Carry = (DigitSum > \$0F) ? 1 : 0. Result = ui16((DigitSum & \$0F) << i). C flag = Carry != 0. Z flag = Result != \$0000. N flag = (Result & \$8000) != 0.
5.2	Read PC:PB	Store as OpCode.
+X.1		

ROR (66)		Cycles: 5	Size: 2
Direct (Read/Modify/Write)			
Cycle	R/W	Desc.	
-X.2	Read PC:PB	Store as OpCode.	
1.1		Inc. PC .	
1.2	Read PC:PB	Store as Operand. If D.L is 0, skip the next cycle.	
2.1		Inc. PC .	
2.2			
3.1		Inc. PC if previous cycle was skipped. Set Address to Operand + D .	
3.2	Read Address	Store as Operand. If M flag is set, skip the next cycle.	
4.1			
4.2	Read Address + 1	Store as Operand.H.	
5.1		If M flag is set, set C based off (Operand.L & \$01), perform Operand.L = (Operand.L >> 1) (C << 7), and set N based off (Operand.L & \$80) and Z based off Operand.L, otherwise set C based off (Operand.L & \$01), perform Operand = (Operand >> 1) (C << 15), and set N based off (Operand.H & \$80) and Z based off Operand.	
5.2		If M flag is set, skip the next cycle.	
6.1			
6.2	Write to Address + 1	Write Operand.H.	
7.1			
7.2	Write to Address	Write Operand.L.	
8.1			
8.2	Read PC:PB	Store as OpCode.	
+X.1			

ADC (67) Cycles: 6		Size: 2
Direct Indirect Long (Read)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Store as Operand. If D.L is 0, skip the next cycle.
2.1		Inc. PC .
2.2		
3.1		Inc. PC if previous cycle was skipped. Set Address to Operand + D .
3.2	Read Address	Store as Pointer.
4.1		
4.2	Read Address + 1	Store as Pointer.H.
5.1		
5.2	Read Address + 2	Store as Bank.
6.1		
6.2	Read Pointer:Bank	Store as Operand. If M flag is set, skip the next cycle.
7.1		
7.2	Read Pointer + 1:Bank (With Carry)	Store as Operand.H.
8.1		If M flag == 0 (8-bit): If D flag == 0: Result = ui16(A.L) + ui16(Operand) + C. V flag = (~ui16(A.L) ^ ui16(Operand)) & (ui16(A.L) ^ ui8(Result)) & \$80) != 0. C flag = Result > \$FF. Result = ui8(Result). Z flag = Result == \$00. N flag = (Result & \$80) != 0. If D flag == 1: Lo = ui16(A.L & 0x0F) + ui16(Operand & 0x0F) + C; if (Lo > 9) Lo += 6. CarryToHi = Lo > \$0F ? 1 : 0. HiSum = ui16(A.L >> 4) + ui16(Operand >> 4) + CarryToHi. V flag = (~((A.L >> 4) ^ (Operand >> 4)) & ((A.L >> 4) ^ HiSum) & \$08) != 0. if (HiSum > 9) HiSum += 6. C flag = HiSum > \$0F. Result = ui8(((HiSum & 0x0F) << 4) (Lo & \$0F)). Z flag = Result == \$00. N flag = (Result & \$80) != 0. If M flag == 0 (16-bit): If D flag == 0: Result = ui32(A) + ui32(Operand) + C. V flag = (~ui32(A) ^ ui32(Operand)) & (ui32(A) ^ ui16(Result)) & \$8000) != 0). C flag = Result > \$FFFF. Result = ui16(Result). Z flag = Result == \$0000. N flag = (Result & \$8000) != 0. If D flag == 1: Result = ui16(0). Carry = ui16(C). for i = 0, 4, 8, 12: DigitSum = ui16((A >> i) & \$0F) + ui16((Operand >> i) & \$0F) + Carry. if (i == 12) V flag = (~(A >> 12) ^ (Operand >> 12)) & ((A >> 12) ^ DigitSum) & \$08) != 0. if (DigitSum > 9) DigitSum += 6. Carry = (DigitSum > \$0F) ? 1 : 0. Result = ui16((DigitSum & \$0F) << i). C flag = Carry != 0. Z flag = Result != \$0000. N flag = (Result & \$8000) != 0.
8.2	Read PC:PB	Store as OpCode.
+X.1		

PLA (68)	Cycles: 4	Size: 1
Implied		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2		
2.1		
2.2		
3.1		Inc. S by 1.
3.2	Read (S +0)	Store as Operand.L. If M flag is set, skip the next cycle.
4.1		
4.2	Read (S +1)	Store as Operand.H.
5.1		Inc. S by 1. If M flag is set, set A.L to Operand.L and set N based off (A.L & \$80) and Z based off A.L , otherwise set A to Operand and set N based off (A.H & \$80) and Z based off A .
5.2	Read PC:PB	Store as OpCode.
+X.1		

ADC (69) Cycles: 2		Size: 2
Immediate		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Store as Operand. If M flag is set, skip the next cycle.
2.1		Inc. PC .
2.2	Read PC:PB	Store as Operand.H.
3.1		If M flag == 0 (8-bit): If D flag == 0: Result = ui16(A.L) + ui16(Operand) + C. V flag = (~ui16(A.L) ^ ui16(Operand)) & (ui16(A.L) ^ ui8(Result)) & \$80) != 0. C flag = Result > \$FF. Result = ui8(Result). Z flag = Result == \$00. N flag = (Result & \$80) != 0. If D flag == 1: Lo = ui16(A.L & 0x0F) + ui16(Operand & 0x0F) + C; if (Lo > 9) Lo += 6. CarryToHi = Lo > \$0F ? 1 : 0. HiSum = ui16(A.L >> 4) + ui16(Operand >> 4) + CarryToHi. V flag = (~((A.L >> 4) ^ (Operand >> 4)) & ((A.L >> 4) ^ HiSum) & \$08) != 0. if (HiSum > 9) HiSum += 6. C flag = HiSum > \$0F. Result = ui8(((HiSum & 0x0F) << 4) (Lo & \$0F)). Z flag = Result == \$00. N flag = (Result & \$80) != 0. If M flag == 0 (16-bit): If D flag == 0: Result = ui32(A) + ui32(Operand) + C. V flag = (~ui32(A) ^ ui32(Operand)) & (ui32(A) ^ ui16(Result)) & \$8000) != 0). C flag = Result > \$FFFF. Result = ui16(Result). Z flag = Result == \$0000. N flag = (Result & \$8000) != 0. If D flag == 1: Result = ui16(0). Carry = ui16(C). for i = 0, 4, 8, 12: DigitSum = ui16((A >> i) & \$0F) + ui16((Operand >> i) & \$0F) + Carry. if (i == 12) V flag = (~(A >> 12) ^ (Operand >> 12)) & ((A >> 12) ^ DigitSum) & \$08) != 0. if (DigitSum > 9) DigitSum += 6. Carry = (DigitSum > \$0F) ? 1 : 0. Result = ui16((DigitSum & \$0F) << i). C flag = Carry != 0. Z flag = Result != \$0000. N flag = (Result & \$8000) != 0.
3.2	Read PC:PB	Store as OpCode.
+X.1		

ROR (6A)	Cycles: 2	Size: 1
Implied		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1	Read PC:PB	Inc. PC .
1.2		Discard.
2.1		If M flag is set, set C based off (A.L & \$01), perform A.L = (A.L >> 1) (C << 7), and set N based off (A.L & \$80) and Z based off A.L , otherwise set C based off (A.L & \$01), perform A = (A >> 1) (C << 15), and set N based off (A.H & \$80) and Z based off A .
2.2		Store as OpCode.
+X.1		

RTL (6B)	Cycles: 6	Size: 1
Implied		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2		
2.1		
2.2		
3.1		
3.2	Read (S +1)	Store as Operand.L.
4.1		
4.2	Read (S +2)	Store as Operand.H.
5.1		
5.2	Read (S +3)	Store as PB .
6.1		Inc. S by 3. Set PC to (Operand + 1). Set PB to Bank.
6.2	Read PC:PB	Store as OpCode.
+X.1		

JMP (6C)		Cycles: 5	Size: 3
Absolute Indirect (Jump)			
Cycle	R/W		Desc.
-X.2	Read PC:PB		Store as OpCode.
1.1			Inc. PC .
1.2	Read PC:PB		Store as Pointer.
2.1			Inc. PC .
2.2	Read PC:PB		Store as Pointer.H.
3.1			Inc. PC .
3.2	Read Pointer		Store as Address.
4.1			
4.2	Read Pointer + 1		Store as Address.H.
5.1			Set PC to Address.
5.2	Read PC:PB		Store as OpCode.
+X.1			

ADC (6D) Cycles: 4		Size: 3
Absolute (Read)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Store as Address.
2.1		Inc. PC .
2.2	Read PC:PB	Store as Address.H.
3.1		Inc. PC .
3.2	Read Address: DB	Store as Operand. If M flag is set, skip the next cycle.
4.1		
4.2	Read Address + 1: DB	Store as Operand.H.
5.1		If M flag == 0 (8-bit): If D flag == 0: Result = ui16(A.L) + ui16(Operand) + C. V flag = (~ui16(A.L) ^ ui16(Operand)) & (ui16(A.L) ^ ui8(Result)) & \$80) != 0. C flag = Result > \$FF. Result = ui8(Result). Z flag = Result == \$00. N flag = (Result & \$80) != 0. If D flag == 1: Lo = ui16(A.L & 0x0F) + ui16(Operand & 0x0F) + C; if (Lo > 9) Lo += 6. CarryToHi = Lo > \$0F ? 1 : 0. HiSum = ui16(A.L >> 4) + ui16(Operand >> 4) + CarryToHi. V flag = (~((A.L >> 4) ^ (Operand >> 4)) & ((A.L >> 4) ^ HiSum) & \$08) != 0. if (HiSum > 9) HiSum += 6. C flag = HiSum > \$0F. Result = ui8(((HiSum & 0x0F) << 4) (Lo & \$0F)). Z flag = Result == \$00. N flag = (Result & \$80) != 0. If M flag == 0 (16-bit): If D flag == 0: Result = ui32(A) + ui32(Operand) + C. V flag = (~ui32(A) ^ ui32(Operand)) & (ui32(A) ^ ui16(Result)) & \$8000) != 0). C flag = Result > \$FFFF. Result = ui16(Result). Z flag = Result == \$0000. N flag = (Result & \$8000) != 0. If D flag == 1: Result = ui16(0). Carry = ui16(C). for i = 0, 4, 8, 12: DigitSum = ui16((A >> i) & \$0F) + ui16((Operand >> i) & \$0F) + Carry. if (i == 12) V flag = (~(A >> 12) ^ (Operand >> 12)) & ((A >> 12) ^ DigitSum) & \$08) != 0. if (DigitSum > 9) DigitSum += 6. Carry = (DigitSum > \$0F) ? 1 : 0. Result = ui16((DigitSum & \$0F) << i). C flag = Carry != 0. Z flag = Result != \$0000. N flag = (Result & \$8000) != 0.
5.2	Read PC:PB	Store as OpCode.
+X.1		

ROR (6E) Cycles: 6		Size: 3
Absolute (Read/Modify/Write)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Store as Address.
2.1		Inc. PC .
2.2	Read PC:PB	Store as Address.H.
3.1		Inc. PC .
3.2	Read Address: DB	Store as Operand. If M flag is set, skip the next cycle.
4.1		
4.2	Read Address + 1: DB	Store as Operand.H.
5.1		If M flag is set, set C based off (Operand.L & \$01), perform Operand.L = (Operand.L >> 1) (C << 7), and set N based off (Operand.L & \$80) and Z based off Operand.L, otherwise set C based off (Operand.L & \$01), perform Operand = (Operand >> 1) (C << 15), and set N based off (Operand.H & \$80) and Z based off Operand.
5.2		If M flag is set, skip the next cycle.
6.1		
6.2	Write to Address + 1: DB	Write Operand.H.
7.1		
7.2	Write to Address: DB	Write Operand.L.
8.1		
8.2	Read PC:PB	Store as OpCode.
+X.1		

ADC (6F) Cycles: 5		Size: 4
Absolute Long (Read)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Store as Address.
2.1		Inc. PC .
2.2	Read PC:PB	Store as Address.H.
3.1		Inc. PC .
3.2	Read PC:PB	Stores as Bank.
4.1		Inc. PC .
4.2	Read Address:Bank	Store as Operand. If M flag is set, skip the next cycle.
5.1		
5.2	Read Address + 1:Bank (With Carry)	Store as Operand.H.
6.1		If M flag == 0 (8-bit): If D flag == 0: Result = ui16(A.L) + ui16(Operand) + C. V flag = (~ui16(A.L) ^ ui16(Operand)) & (ui16(A.L) ^ ui8(Result)) & \$80) != 0. C flag = Result > \$FF. Result = ui8(Result). Z flag = Result == \$00. N flag = (Result & \$80) != 0. If D flag == 1: Lo = ui16(A.L & 0x0F) + ui16(Operand & 0x0F) + C; if (Lo > 9) Lo += 6. CarryToHi = Lo > \$0F ? 1 : 0. HiSum = ui16(A.L >> 4) + ui16(Operand >> 4) + CarryToHi. V flag = (~((A.L >> 4) ^ (Operand >> 4)) & ((A.L >> 4) ^ HiSum) & \$08) != 0. if (HiSum > 9) HiSum += 6. C flag = HiSum > \$0F. Result = ui8(((HiSum & 0x0F) << 4) (Lo & \$0F)). Z flag = Result == \$00. N flag = (Result & \$80) != 0. If M flag == 0 (16-bit): If D flag == 0: Result = ui32(A) + ui32(Operand) + C. V flag = (~ui32(A) ^ ui32(Operand)) & (ui32(A) ^ ui16(Result)) & \$8000) != 0). C flag = Result > \$FFFF. Result = ui16(Result). Z flag = Result == \$0000. N flag = (Result & \$8000) != 0. If D flag == 1: Result = ui16(0). Carry = ui16(C). for i = 0, 4, 8, 12: DigitSum = ui16((A >> i) & \$0F) + ui16((Operand >> i) & \$0F) + Carry. if (i == 12) V flag = (~(A >> 12) ^ (Operand >> 12)) & ((A >> 12) ^ DigitSum) & \$08) != 0. if (DigitSum > 9) DigitSum += 6. Carry = (DigitSum > \$0F) ? 1 : 0. Result = ui16((DigitSum & \$0F) << i). C flag = Carry != 0. Z flag = Result != \$0000. N flag = (Result & \$8000) != 0.
6.2	Read PC:PB	Store as OpCode.
+X.1		

BVS (70)	Cycles: 2	Size: 2
Relative		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC . Decide to branch if V is 1.
1.2	Read PC:PB	Store as Operand. Address = i16(i8(Operand.L)) + PC . If not branching, poll interrupts.
2.1		Inc. PC . If not branching, end (next half-cycle is 3.2).
2.2		Poll interrupts.
3.1		Set PC to Address.
3.2	Read PC:PB	Store as OpCode.
+X.1		

ADC (71) Cycles: 5 Size: 2		
Direct Indirect Indexed (d),y (Read)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Store as Operand. If D.L is 0, skip the next cycle.
2.1		Inc. PC .
2.2		
3.1		Inc. PC if previous cycle was skipped. Set Address to Operand + D .
3.2	Read Address	Store as Pointer.
4.1		
4.2	Read Address + 1	Store as Pointer.H.
5.1		Perform Orig = Pointer. Tmp = ui32(Pointer + Y). Pointer = ui16(Tmp). If Orig.H == Pointer.H and the X flag is set, skip 2 half-cycles. Bank = DB + (Tmp >> 16).
5.2		
6.1		
6.2	Read Pointer:Bank	Store as Operand. If M flag is set, skip the next cycle.
7.1		
7.2	Read Pointer + 1:Bank (With Carry)	Store as Operand.H.
8.1		If M flag == 0 (8-bit): If D flag == 0: Result = ui16(A.L) + ui16(Operand) + C. V flag = (~ui16(A.L) ^ ui16(Operand)) & (ui16(A.L) ^ ui8(Result)) & \$80 != 0. C flag = Result > \$FF. Result = ui8(Result). Z flag = Result == \$00. N flag = (Result & \$80) != 0. If D flag == 1: Lo = ui16(A.L & 0x0F) + ui16(Operand & 0x0F) + C; if (Lo > 9) Lo += 6. CarryToHi = Lo > \$0F ? 1 : 0. HiSum = ui16(A.L >> 4) + ui16(Operand >> 4) + CarryToHi. V flag = (~((A.L >> 4) ^ (Operand >> 4)) & ((A.L >> 4) ^ HiSum) & \$08) != 0. if (HiSum > 9) HiSum += 6. C flag = HiSum > \$0F. Result = ui8(((HiSum & 0x0F) << 4) (Lo & \$0F)). Z flag = Result == \$00. N flag = (Result & \$80) != 0. If M flag == 0 (16-bit): If D flag == 0: Result = ui32(A) + ui32(Operand) + C. V flag = (~ui32(A) ^ ui32(Operand)) & (ui32(A) ^ ui16(Result)) & \$8000 != 0. C flag = Result > \$FFFF. Result = ui16(Result). Z flag = Result == \$0000. N flag = (Result & \$8000) != 0. If D flag == 1: Result = ui16(0). Carry = ui16(C). for i = 0, 4, 8, 12: DigitSum = ui16((A >> i) & \$0F) + ui16((Operand >> i) & \$0F) + Carry. if (i == 12) V flag = (~(A >> 12) ^ (Operand >> 12)) & ((A >> 12) ^ DigitSum) & \$08 != 0. if (DigitSum > 9) DigitSum += 6. Carry = (DigitSum > \$0F) ? 1 : 0. Result = ui16((DigitSum & \$0F) << i). C flag = Carry != 0. Z flag = Result != \$0000. N flag = (Result & \$8000) != 0.
8.2	Read PC:PB	Store as OpCode.
+X.1		

ADC (72)	Cycles: 5	Size: 2
Direct Indirect (Read)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Store as Operand. If D.L is 0, skip the next cycle.
2.1		Inc. PC .
2.2		
3.1		Inc. PC if previous cycle was skipped. Set Address to Operand + D .
3.2	Read Address	Store as Pointer.
4.1		
4.2	Read Address + 1	Store as Pointer.H.
5.1		
5.2	Read Pointer: DB	Store as Operand. If M flag is set, skip the next cycle.
6.1		
6.2	Read Pointer + 1: DB	Store as Operand.H.
7.1		If M flag == 0 (8-bit): If D flag == 0: Result = ui16(A.L) + ui16(Operand) + C. V flag = (~ui16(A.L) ^ ui16(Operand)) & (ui16(A.L) ^ ui8(Result)) & \$80 != 0. C flag = Result > \$FF. Result = ui8(Result). Z flag = Result == \$00. N flag = (Result & \$80) != 0. If D flag == 1: Lo = ui16(A.L & 0x0F) + ui16(Operand & 0x0F) + C; if (Lo > 9) Lo += 6. CarryToHi = Lo > \$0F ? 1 : 0. HiSum = ui16(A.L >> 4) + ui16(Operand >> 4) + CarryToHi. V flag = (~((A.L >> 4) ^ (Operand >> 4)) & ((A.L >> 4) ^ HiSum) & \$08) != 0. if (HiSum > 9) HiSum += 6. C flag = HiSum > \$0F. Result = ui8(((HiSum & 0x0F) << 4) (Lo & \$0F)). Z flag = Result == \$00. N flag = (Result & \$80) != 0. If M flag == 0 (16-bit): If D flag == 0: Result = ui32(A) + ui32(Operand) + C. V flag = (~ui32(A) ^ ui32(Operand)) & (ui32(A) ^ ui16(Result)) & \$8000 != 0. C flag = Result > \$FFFF. Result = ui16(Result). Z flag = Result == \$0000. N flag = (Result & \$8000) != 0. If D flag == 1: Result = ui16(0). Carry = ui16(C). for i = 0, 4, 8, 12: DigitSum = ui16((A >> i) & \$0F) + ui16((Operand >> i) & \$0F) + Carry. if (i == 12) V flag = (~(A >> 12) ^ (Operand >> 12)) & ((A >> 12) ^ DigitSum) & \$08 != 0. if (DigitSum > 9) DigitSum += 6. Carry = (DigitSum > \$0F) ? 1 : 0. Result = ui16((DigitSum & \$0F) << i). C flag = Carry != 0. Z flag = Result != \$0000. N flag = (Result & \$8000) != 0.
7.2	Read PC:PB	Store as OpCode.
+X.1		

ADC (73) Cycles: 7		Size: 2
Stack Relative Indirect Indexed (Read)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Store as Pointer.
2.1		Inc. PC . Set Address to Pointer + S .
2.2		
3.1		
3.2	Read Address	Store as Pointer.
4.1		
4.2	Read Address + 1	Store as Pointer.H.
5.1		Perform $\text{Tmp} = \text{ui32}(\text{Pointer} + \mathbf{Y})$. Pointer = $\text{ui16}(\text{Tmp})$. Bank = $\mathbf{DB} + (\text{Tmp} \gg 16)$.
5.2		
6.1		
6.2	Read Pointer:Bank	Store as Operand. If M flag is set, skip the next cycle.
7.1		
7.2	Read Pointer + 1:Bank (With Carry)	Store as Operand.H.
8.1		If M flag == 0 (8-bit): If D flag == 0: Result = $\text{ui16}(\mathbf{A.L}) + \text{ui16}(\text{Operand}) + \mathbf{C}$. V flag = $(\sim(\text{ui16}(\mathbf{A.L}) \wedge \text{ui16}(\text{Operand})) \& (\text{ui16}(\mathbf{A.L}) \wedge \text{ui8}(\text{Result})) \& \\$80) \neq 0$. C flag = Result > \$FF. Result = $\text{ui8}(\text{Result})$. Z flag = Result == \$00. N flag = $(\text{Result} \& \\$80) \neq 0$. If D flag == 1: Lo = $\text{ui16}(\mathbf{A.L} \& 0x0F) + \text{ui16}(\text{Operand} \& 0x0F) + \mathbf{C}$; if (Lo > 9) Lo += 6. CarryToHi = Lo > \$0F ? 1 : 0. HiSum = $\text{ui16}(\mathbf{A.L} \gg 4) + \text{ui16}(\text{Operand} \gg 4) + \text{CarryToHi}$. V flag = $(\sim((\mathbf{A.L} \gg 4) \wedge (\text{Operand} \gg 4)) \& ((\mathbf{A.L} \gg 4) \wedge \text{HiSum}) \& \\$08) \neq 0$. if (HiSum > 9) HiSum += 6. C flag = HiSum > \$0F. Result = $\text{ui8}(((\text{HiSum} \& 0x0F) << 4) \mid (\text{Lo} \& \\$0F))$. Z flag = Result == \$00. N flag = $(\text{Result} \& \\$80) \neq 0$. If M flag == 0 (16-bit): If D flag == 0: Result = $\text{ui32}(\mathbf{A}) + \text{ui32}(\text{Operand}) + \mathbf{C}$. V flag = $(\sim(\text{ui32}(\mathbf{A}) \wedge \text{ui32}(\text{Operand})) \& (\text{ui32}(\mathbf{A}) \wedge \text{ui16}(\text{Result})) \& \\$8000) \neq 0$. C flag = Result > \$FFFF. Result = $\text{ui16}(\text{Result})$. Z flag = Result == \$0000. N flag = $(\text{Result} \& \\$8000) \neq 0$. If D flag == 1: Result = $\text{ui16}(0)$. Carry = $\text{ui16}(\mathbf{C})$. for i = 0, 4, 8, 12: DigitSum = $\text{ui16}((\mathbf{A} \gg i) \& \\$0F) + \text{ui16}((\text{Operand} \gg i) \& \\$0F) + \text{Carry}$. if (i == 12) V flag = $(\sim((\mathbf{A} \gg 12) \wedge (\text{Operand} \gg 12)) \& ((\mathbf{A} \gg 12) \wedge \text{DigitSum}) \& \\$08) \neq 0$. if (DigitSum > 9) DigitSum += 6. Carry = $(\text{DigitSum} > \\$0F) ? 1 : 0$. Result = $\text{ui16}((\text{DigitSum} \& \\$0F) << i)$. C flag = Carry != 0. Z flag = Result != \$0000. N flag = $(\text{Result} \& \\$8000) \neq 0$.
8.2	Read PC:PB	Store as OpCode.
+X.1		

STZ (74)	Cycles: 4	Size: 2
Direct, X (Write)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1	Read PC:PB	Inc. PC .
1.2		Store as Operand.
2.1		Inc. PC .
2.2		
3.1		Address = Operand + X + D . If D.L is 0, skip 2 half-cycles.
3.2		
4.1		
4.2	Write to Address	Write 0. If M flag is set, skip the next cycle.
5.1	Write to Address + 1	
5.2		Write 0.
6.1	Read PC:PB	
6.2		Store as OpCode.
+X.1		

ADC (75) Cycles: 4		Size: 2
Direct, X (Read)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1	Read PC:PB	Inc. PC .
1.2		Store as Operand.
2.1		Inc. PC .
2.2		
3.1		Address = Operand + X + D . If D.L is 0, skip 2 half-cycles.
3.2		
4.1		
4.2	Read Address	Store as Operand. If M flag is set, skip the next cycle.
5.1		
5.2	Read Address + 1	Store as Operand.H.
6.1		If M flag == 0 (8-bit): If D flag == 0: Result = ui16(A.L) + ui16(Operand) + C. V flag = (~ui16(A.L) ^ ui16(Operand)) & (ui16(A.L) ^ ui8(Result)) & \$80) != 0. C flag = Result > \$FF. Result = ui8(Result). Z flag = Result == \$00. N flag = (Result & \$80) != 0. If D flag == 1: Lo = ui16(A.L & 0x0F) + ui16(Operand & 0x0F) + C; if (Lo > 9) Lo += 6. CarryToHi = Lo > \$0F ? 1 : 0. HiSum = ui16(A.L >> 4) + ui16(Operand >> 4) + CarryToHi. V flag = (~((A.L >> 4) ^ (Operand >> 4)) & ((A.L >> 4) ^ HiSum) & \$08) != 0. if (HiSum > 9) HiSum += 6. C flag = HiSum > \$0F. Result = ui8(((HiSum & 0x0F) << 4) (Lo & \$0F)). Z flag = Result == \$00. N flag = (Result & \$80) != 0. If M flag == 0 (16-bit): If D flag == 0: Result = ui32(A) + ui32(Operand) + C. V flag = (~ui32(A) ^ ui32(Operand)) & (ui32(A) ^ ui16(Result)) & \$8000) != 0). C flag = Result > \$FFFF. Result = ui16(Result). Z flag = Result == \$0000. N flag = (Result & \$8000) != 0. If D flag == 1: Result = ui16(0). Carry = ui16(C). for i = 0, 4, 8, 12: DigitSum = ui16((A >> i) & \$0F) + ui16((Operand >> i) & \$0F) + Carry. if (i == 12) V flag = (~(A >> 12) ^ (Operand >> 12)) & ((A >> 12) ^ DigitSum) & \$08) != 0. if (DigitSum > 9) DigitSum += 6. Carry = (DigitSum > \$0F) ? 1 : 0. Result = ui16((DigitSum & \$0F) << i). C flag = Carry != 0. Z flag = Result != \$0000. N flag = (Result & \$8000) != 0.
6.2	Read PC:PB	Store as OpCode.
+X.1		

ROR (76)		Cycles: 6	Size: 2
Direct, X (Read/Modify/Write)			
Cycle	R/W	Desc.	
-X.2	Read PC:PB	Store as OpCode.	
1.1	Read PC:PB	Inc. PC .	
1.2		Store as Operand.	
2.1		Inc. PC .	
2.2			
3.1		Address = Operand + X + D . If D.L is 0, skip 2 half-cycles.	
3.2			
4.1			
4.2		Read Address	
5.1	Read Address + 1	Store as Operand. If M flag is set, skip the next cycle.	
5.2		Store as Operand.H.	
6.1		If M flag is set, set C based off (Operand.L & \$01), perform Operand.L = (Operand.L >> 1) (C << 7), and set N based off (Operand.L & \$80) and Z based off Operand.L, otherwise set C based off (Operand.L & \$01), perform Operand = (Operand >> 1) (C << 15), and set N based off (Operand.H & \$80) and Z based off Operand.	
6.2		If M flag is set, skip the next cycle.	
7.1	Write to Address + 1		
7.2		Write Operand.H.	
8.1	Write to Address		
8.2		Write Operand.L.	
9.1			
9.2	Read PC:PB	Store as OpCode.	
+X.1			

ADC (77) Cycles: 6		Size: 2
Direct Indirect Indexed Long [d],y (Read)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Store as Operand. If D.L is 0, skip the next cycle.
2.1		Inc. PC .
2.2		
3.1		Inc. PC if previous cycle was skipped. Set Address to Operand + D .
3.2	Read Address	Store as Pointer.
4.1		
4.2	Read Address + 1	Store as Pointer.H.
5.1		
5.2	Read Address + 2	Store as Bank.
6.1		Perform $\text{Tmp} = \text{ui32}(\text{Pointer} + \mathbf{Y})$. Pointer = $\text{ui16}(\text{Tmp})$. Bank += $(\text{Tmp} \gg 16)$.
6.2	Read Pointer:Bank	Store as Operand. If M flag is set, skip the next cycle.
7.1		
7.2	Read Pointer + 1:Bank (With Carry)	Store as Operand.H.
8.1		If M flag == 0 (8-bit): If D flag == 0: Result = $\text{ui16}(\mathbf{A.L}) + \text{ui16}(\text{Operand}) + \mathbf{C}$. V flag = $(\sim(\text{ui16}(\mathbf{A.L}) \wedge \text{ui16}(\text{Operand})) \& (\text{ui16}(\mathbf{A.L}) \wedge \text{ui8}(\text{Result})) \& \\$80) \neq 0$. C flag = Result > \$FF. Result = $\text{ui8}(\text{Result})$. Z flag = Result == \$00. N flag = $(\text{Result} \& \\$80) \neq 0$. If D flag == 1: Lo = $\text{ui16}(\mathbf{A.L} \& 0x0F) + \text{ui16}(\text{Operand} \& 0x0F) + \mathbf{C}$; if (Lo > 9) Lo += 6. CarryToHi = Lo > \$0F ? 1 : 0. HiSum = $\text{ui16}(\mathbf{A.L} \gg 4) + \text{ui16}(\text{Operand} \gg 4) + \text{CarryToHi}$. V flag = $(\sim((\mathbf{A.L} \gg 4) \wedge (\text{Operand} \gg 4)) \& ((\mathbf{A.L} \gg 4) \wedge \text{HiSum}) \& \\$08) \neq 0$. if (HiSum > 9) HiSum += 6. C flag = HiSum > \$0F. Result = $\text{ui8}(((\text{HiSum} \& 0x0F) << 4) \mid (\text{Lo} \& \\$0F))$. Z flag = Result == \$00. N flag = $(\text{Result} \& \\$80) \neq 0$. If M flag == 0 (16-bit): If D flag == 0: Result = $\text{ui32}(\mathbf{A}) + \text{ui32}(\text{Operand}) + \mathbf{C}$. V flag = $(\sim(\text{ui32}(\mathbf{A}) \wedge \text{ui32}(\text{Operand})) \& (\text{ui32}(\mathbf{A}) \wedge \text{ui16}(\text{Result})) \& \\$8000) \neq 0$. C flag = Result > \$FFFF. Result = $\text{ui16}(\text{Result})$. Z flag = Result == \$0000. N flag = $(\text{Result} \& \\$8000) \neq 0$. If D flag == 1: Result = $\text{ui16}(0)$. Carry = $\text{ui16}(\mathbf{C})$. for i = 0, 4, 8, 12: DigitSum = $\text{ui16}((\mathbf{A} \gg i) \& \\$0F) + \text{ui16}((\text{Operand} \gg i) \& \\$0F) + \text{Carry}$. if (i == 12) V flag = $(\sim((\mathbf{A} \gg 12) \wedge (\text{Operand} \gg 12)) \& ((\mathbf{A} \gg 12) \wedge \text{DigitSum}) \& \\$08) \neq 0$. if (DigitSum > 9) DigitSum += 6. Carry = $(\text{DigitSum} > \\$0F) ? 1 : 0$. Result = $\text{ui16}((\text{DigitSum} \& \\$0F) << i)$. C flag = Carry != 0. Z flag = Result != \$0000. N flag = $(\text{Result} \& \\$8000) \neq 0$.
8.2	Read PC:PB	Store as OpCode.
+X.1		

CLI (78)	Cycles: 2	Size: 1
Implied		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Discard.
2.1		Sets the I flag to 1.
2.2	Read PC:PB	Store as OpCode.
+X.1		

ADC (79) Cycles: 4		Size: 3
Absolute, Y (Read)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Store as Pointer.
2.1		Inc. PC .
2.2	Read PC:PB	Store as Pointer.H.
3.1		Inc. PC . Perform Orig = Pointer. Tmp = ui32(Pointer + Y). Pointer = ui16(Tmp). If Orig.H == Pointer.H and the X flag is set, skip 2 half-cycles. Bank = DB + (Tmp >> 16).
3.2		
4.1		
4.2	Read Pointer:Bank	Store as Operand. If M flag is set, skip the next cycle.
5.1		
5.2	Read Pointer + 1:Bank (With Carry)	Store as Operand.H.
6.1		If M flag == 0 (8-bit): If D flag == 0: Result = ui16(A.L) + ui16(Operand) + C. V flag = (~ui16(A.L) ^ ui16(Operand)) & (ui16(A.L) ^ ui8(Result)) & \$80) != 0. C flag = Result > \$FF. Result = ui8(Result). Z flag = Result == \$00. N flag = (Result & \$80) != 0. If D flag == 1: Lo = ui16(A.L & 0x0F) + ui16(Operand & 0x0F) + C; if (Lo > 9) Lo += 6. CarryToHi = Lo > \$0F ? 1 : 0. HiSum = ui16(A.L >> 4) + ui16(Operand >> 4) + CarryToHi. V flag = (~((A.L >> 4) ^ (Operand >> 4)) & ((A.L >> 4) ^ HiSum) & \$08) != 0. if (HiSum > 9) HiSum += 6. C flag = HiSum > \$0F. Result = ui8(((HiSum & 0x0F) << 4) (Lo & \$0F)). Z flag = Result == \$00. N flag = (Result & \$80) != 0. If M flag == 0 (16-bit): If D flag == 0: Result = ui32(A) + ui32(Operand) + C. V flag = (~ui32(A) ^ ui32(Operand)) & (ui32(A) ^ ui16(Result)) & \$8000) != 0). C flag = Result > \$FFFF. Result = ui16(Result). Z flag = Result == \$0000. N flag = (Result & \$8000) != 0. If D flag == 1: Result = ui16(0). Carry = ui16(C). for i = 0, 4, 8, 12: DigitSum = ui16((A >> i) & \$0F) + ui16((Operand >> i) & \$0F) + Carry. if (i == 12) V flag = (~(A >> 12) ^ (Operand >> 12)) & ((A >> 12) ^ DigitSum) & \$08) != 0. if (DigitSum > 9) DigitSum += 6. Carry = (DigitSum > \$0F) ? 1 : 0. Result = ui16((DigitSum & \$0F) << i). C flag = Carry != 0. Z flag = Result != \$0000. N flag = (Result & \$8000) != 0.
6.2	Read PC:PB	Store as OpCode.
+X.1		

PLY (7A)		Cycles: 4	Size: 1
Implied			
Cycle	R/W		Desc.
-X.2	Read PC:PB		Store as OpCode.
1.1			Inc. PC .
1.2			
2.1			
2.2			
3.1			Inc. S by 1.
3.2	Read (S +0)		Store as Operand.L. If X flag is set, skip the next cycle.
4.1			
4.2	Read (S +1)		Store as Operand.H.
5.1			Inc. S by 1. If X flag is set, set Y .L to Operand.L, set N based off (Y .L & \$80), and Z based off Y .L, otherwise set Y to Operand, set N based off (Y .H & \$80), and Z based off Y .
5.2	Read PC:PB		Store as OpCode.
+X.1			

TDC (7B)	Cycles: 2	Size: 1
Implied		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Discard.
2.1		Set A to D . Set N based off (A .H & \$80) and Z based off A .
2.2	Read PC:PB	Store as OpCode.
+X.1		

JMP (7C)		Cycles: 6	Size: 3
Absolute Indexed Indirect (Jump)			
Cycle	R/W	Desc.	
-X.2	Read PC:PB	Store as OpCode.	
1.1		Inc. PC .	
1.2	Read PC:PB	Store as Pointer.	
2.1		Inc. PC .	
2.2	Read PC:PB	Store as Pointer.H.	
3.1		Perform Pointer += X . Bank = PB .	
3.2			
4.1			
4.2	Read Pointer:Bank	Store as Address.L.	
5.1			
5.2	Read Pointer + 1:Bank	Store as Address.H.	
6.1		Set PC to Address.	
6.2	Read PC:PB	Store as OpCode.	
+X.1			

ADC (7D) Cycles: 4		Size: 3
Absolute, X (Read)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Store as Pointer.
2.1		Inc. PC .
2.2	Read PC:PB	Store as Pointer.H.
3.1		Inc. PC . Perform Orig = Pointer. Tmp = ui32(Pointer + X). Pointer = ui16(Tmp). If Orig.H == Pointer.H and the X flag is set, skip 2 half-cycles. Bank = DB + (Tmp >> 16).
3.2		
4.1		
4.2	Read Pointer:Bank	Store as Operand. If M flag is set, skip the next cycle.
5.1		
5.2	Read Pointer + 1:Bank (With Carry)	Store as Operand.H.
6.1		If M flag == 0 (8-bit): If D flag == 0: Result = ui16(A.L) + ui16(Operand) + C. V flag = (~ui16(A.L) ^ ui16(Operand)) & (ui16(A.L) ^ ui8(Result)) & \$80) != 0. C flag = Result > \$FF. Result = ui8(Result). Z flag = Result == \$00. N flag = (Result & \$80) != 0. If D flag == 1: Lo = ui16(A.L & 0x0F) + ui16(Operand & 0x0F) + C; if (Lo > 9) Lo += 6. CarryToHi = Lo > \$0F ? 1 : 0. HiSum = ui16(A.L >> 4) + ui16(Operand >> 4) + CarryToHi. V flag = (~((A.L >> 4) ^ (Operand >> 4)) & ((A.L >> 4) ^ HiSum) & \$08) != 0. if (HiSum > 9) HiSum += 6. C flag = HiSum > \$0F. Result = ui8(((HiSum & 0x0F) << 4) (Lo & \$0F)). Z flag = Result == \$00. N flag = (Result & \$80) != 0. If M flag == 0 (16-bit): If D flag == 0: Result = ui32(A) + ui32(Operand) + C. V flag = (~ui32(A) ^ ui32(Operand)) & (ui32(A) ^ ui16(Result)) & \$8000) != 0). C flag = Result > \$FFFF. Result = ui16(Result). Z flag = Result == \$0000. N flag = (Result & \$8000) != 0. If D flag == 1: Result = ui16(0). Carry = ui16(C). for i = 0, 4, 8, 12: DigitSum = ui16((A >> i) & \$0F) + ui16((Operand >> i) & \$0F) + Carry. if (i == 12) V flag = (~(A >> 12) ^ (Operand >> 12)) & ((A >> 12) ^ DigitSum) & \$08) != 0. if (DigitSum > 9) DigitSum += 6. Carry = (DigitSum > \$0F) ? 1 : 0. Result = ui16((DigitSum & \$0F) << i). C flag = Carry != 0. Z flag = Result != \$0000. N flag = (Result & \$8000) != 0.
6.2	Read PC:PB	Store as OpCode.
+X.1		

ROR (7E) Cycles: 7		Size: 3
Absolute, X (Read/Modiy/Write)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Store as Pointer.
2.1		Inc. PC .
2.2	Read PC:PB	Store as Pointer.H.
3.1		Inc. PC . Perform $\text{Tmp} = \text{ui32}(\text{Pointer} + \text{X})$. Pointer = $\text{ui16}(\text{Tmp})$. Bank = $\text{DB} + (\text{Tmp} \gg 16)$.
3.2		
4.1		
4.2	Read Pointer:Bank	Store as Operand. If M flag is set, skip the next cycle.
5.1		
5.2	Read Pointer + 1:Bank (With Carry)	Store as Operand.H.
6.1		If M flag is set, set C based off (Operand.L & \$01), perform $\text{Operand.L} = (\text{Operand.L} \gg 1) \mid (\text{C} \ll 7)$, and set N based off (Operand.L & \$80) and Z based off Operand.L, otherwise set C based off (Operand.L & \$01), perform $\text{Operand} = (\text{Operand} \gg 1) \mid (\text{C} \ll 15)$, and set N based off (Operand.H & \$80) and Z based off Operand.
6.2		If M flag is set, skip the next cycle.
7.1		
7.2	Write to Pointer + 1:Bank (With Carry)	Write Operand.H.
8.1		
8.2	Write to Pointer:Bank	Write Operand.L.
9.1		
9.2	Read PC:PB	Store as OpCode.
+X.1		

ADC (7F) Cycles: 5		Size: 4
Absolute Long, X (Read)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Store as Pointer.
2.1		Inc. PC .
2.2	Read PC:PB	Store as Pointer.H.
3.1		Inc. PC .
3.2	Read PC:PB	Stores as Bank.
4.1		Inc. PC . Perform $\text{Tmp} = \text{ui32}(\text{Pointer} + \text{X})$. Pointer = $\text{ui16}(\text{Tmp})$. Bank += $(\text{Tmp} \gg 16)$.
4.2	Read Pointer:Bank	Store as Operand. If M flag is set, skip the next cycle.
5.1		
5.2	Read Pointer + 1:Bank (With Carry)	Store as Operand.H.
6.1		If M flag == 0 (8-bit): If D flag == 0: Result = $\text{ui16}(\text{A.L}) + \text{ui16}(\text{Operand}) + \text{C}$. V flag = $(\sim(\text{ui16}(\text{A.L}) \wedge \text{ui16}(\text{Operand})) \& (\text{ui16}(\text{A.L}) \wedge \text{ui8}(\text{Result})) \& \\$80) \neq 0$. C flag = $\text{Result} > \\$\text{FF}$. Result = $\text{ui8}(\text{Result})$. Z flag = $\text{Result} == \\$00$. N flag = $(\text{Result} \& \\$80) \neq 0$. If D flag == 1: Lo = $\text{ui16}(\text{A.L} \& 0\text{x0F}) + \text{ui16}(\text{Operand} \& 0\text{x0F}) + \text{C}$; if $(\text{Lo} > 9)$ Lo += 6. CarryToHi = $\text{Lo} > \\$0\text{F} ? 1 : 0$. HiSum = $\text{ui16}(\text{A.L} \gg 4) + \text{ui16}(\text{Operand} \gg 4) + \text{CarryToHi}$. V flag = $(\sim((\text{A.L} \gg 4) \wedge (\text{Operand} \gg 4)) \& ((\text{A.L} \gg 4) \wedge \text{HiSum}) \& \\$08) \neq 0$. if $(\text{HiSum} > 9)$ HiSum += 6. C flag = $\text{HiSum} > \\$0\text{F}$. Result = $\text{ui8}(((\text{HiSum} \& 0\text{x0F}) << 4) \mid (\text{Lo} \& \\$0\text{F}))$. Z flag = $\text{Result} == \\$00$. N flag = $(\text{Result} \& \\$80) \neq 0$. If M flag == 0 (16-bit): If D flag == 0: Result = $\text{ui32}(\text{A}) + \text{ui32}(\text{Operand}) + \text{C}$. V flag = $(\sim(\text{ui32}(\text{A}) \wedge \text{ui32}(\text{Operand})) \& (\text{ui32}(\text{A}) \wedge \text{ui16}(\text{Result})) \& \\$8000) \neq 0$. C flag = $\text{Result} > \\$\text{FFFF}$. Result = $\text{ui16}(\text{Result})$. Z flag = $\text{Result} == \\$0000$. N flag = $(\text{Result} \& \\$8000) \neq 0$. If D flag == 1: Result = $\text{ui16}(0)$. Carry = $\text{ui16}(\text{C})$. for $i = 0, 4, 8, 12$: DigitSum = $\text{ui16}((\text{A} \gg i) \& \\$0\text{F}) + \text{ui16}((\text{Operand} \gg i) \& \\$0\text{F}) + \text{Carry}$. if $(i == 12)$ V flag = $(\sim((\text{A} \gg 12) \wedge (\text{Operand} \gg 12)) \& ((\text{A} \gg 12) \wedge \text{DigitSum}) \& \\$08) \neq 0$. if $(\text{DigitSum} > 9)$ DigitSum += 6. Carry = $(\text{DigitSum} > \\$0\text{F}) ? 1 : 0$. Result = $\text{ui16}((\text{DigitSum} \& \\$0\text{F}) << i)$. C flag = $\text{Carry} \neq 0$. Z flag = $\text{Result} \neq \\$0000$. N flag = $(\text{Result} \& \\$8000) \neq 0$.
6.2	Read PC:PB	Store as OpCode.
+X.1		

BRA (80)	Cycles: 3	Size: 2
Relative		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC . Decide to branch always.
1.2	Read PC:PB	Store as Operand. Address = i16(i8(Operand.L)) + PC . If not branching, poll interrupts.
2.1		Inc. PC . If not branching, end (next half-cycle is 3.2).
2.2		Poll interrupts.
3.1		Set PC to Address.
3.2	Read PC:PB	Store as OpCode.
+X.1		